# Module 4: Divide and Conquer

Serge Gaspers (K17 506)

Course Admins: cs3821@unsw.edu.au
(Liam, Gerald, Song)

School of Computer Science and Engineering
UNSW Sydney

Term 3, 2025

# Table of Contents



1. Introductory Examples

2. Recurrences
2.1 Framework
2.2 Master Theorem

3. Integer Multiplication
3.1 Applying D&C to multiplication of large integers
3.2 The Karatsuba trick

4. Convolutions
4.1 Polynomials
4.2 The Fast Fourier Transform

5. Self-study: Sorting and selection
5.1 Quicksort and quickselect
5.2 Median of medians

6. Puzzle

## Problem

We are given 27 coins of the same denomination; we know that one of them is counterfeit and that it is lighter than the others. Find the counterfeit coin by weighing coins on a pan balance only three times.

## Hint

You can reduce the search space by a third in one weighing!

## Solution

- Divide the coins into three groups of nine, say $A$, $B$ and $C$.
- Weigh group $A$ against group $B$.
  - If one group is lighter than the other, it contains the counterfeit coin.
  - If instead both groups have equal weight, then group $C$ contains the counterfeit coin!
- Repeat with three groups of three, then three groups of one.

# Divide and Conquer



- This method is called "divide-and-conquer".
- You have already seen some prototypical "serious" algorithms designed using this method: binary search, merge sort, quicksort
- We'll now review these algorithms from a divide-and-conquer perspective, and adapt them to solve problems.

# Binary search



- Steps:
  - **Divide:** Test the midpoint of the search range ($\Theta(1)$)
  - **Conquer:** Search one side of the midpoint recursively
  - **Combine:** Pass the answer up the recursion tree ($\Theta(1)$)
- Recursion is $\log_2 n$ levels deep, with a total of $\Theta(1)$ time spent in each level.
- Time complexity is $\Theta(\log n)$.

# Binary search extensions



Given an array $A$, sorted in non-decreasing order, and an integer $x$.

- Lower bound: find the smallest index $i$ such that $A[i] \geq x$.
- Upper bound: find the largest index $i$ such that $A[i] \leq x$.
- Equal range: find the range of indices $\ell..r$ such that $A[\ell] = \ldots = A[r] = x$.

# Example optimisation problem: Maximum Median



## Definition

Recall that the median of an array is the <u>middle value</u> in sorted order.
For example, the median of $[1, 2, 2, 4, 5]$ is 2.

## Problem

Ling has an array $A$ consisting of $2n - 1$ integers, sorted from smallest to largest. Note that the median is initially $A[n]$.

She wants her numbers to grow big and strong, so for each of the following $k$ days she adds 1 to one of her numbers.

At the end of the $k$ days, what is the largest possible median that her array can have?

**Example**

If the starting array is $[1, 2, 2, 4, 5]$ and $k = 3$, the maximum median is 4, achieved by adding 1 to the third number on the first two days and to any number on the last day.

Using all three increases on the third number does not make the median value 5.

- Which numbers should we increase, and how many times?

**Observation**

We are only concerned with the middle number and everything after it. Numbers from the first half of the array can be left small.

- But we still don't know how much to increase the numbers in the second half!
- What if we did know? What if we had a target to aim for, and we just had to report whether it was possible or not?

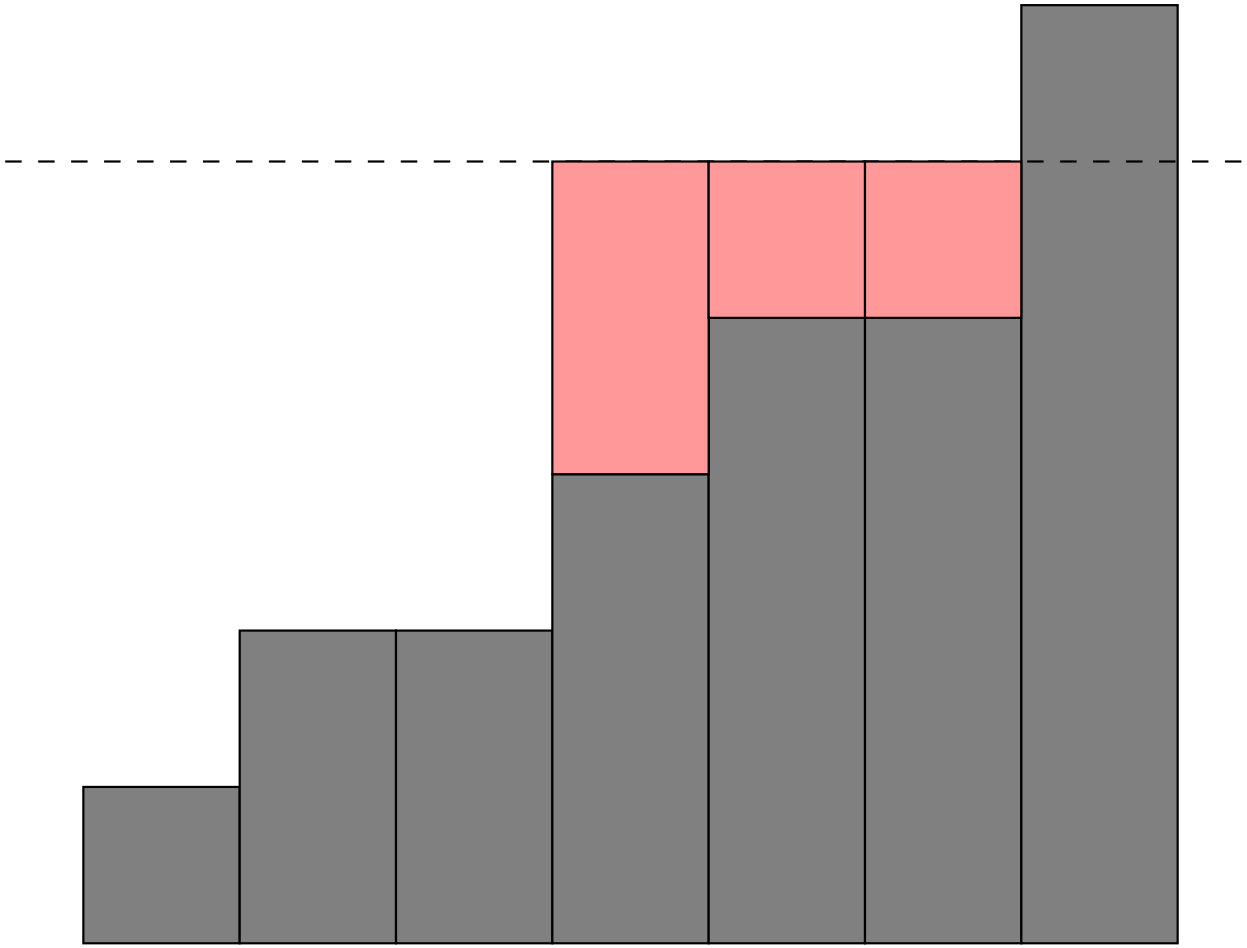
**Related decision problem**

Given positive integers $t$ and $k$. Ling wants to know whether she can make the median of her numbers at least the target value $t$ (by adding 1 to one of her numbers each day for $k$ days).

- How do we get the median to at least $t$?
  - Increase the middle value $A[n]$ and everything after it to $t$; nothing to do for any values that are $\geq t$ already.

# Example decision problem: Maximum Median





# Example decision problem: Maximum Median



- To increase the median to at least $t$, we will need to increase the values:
  - at index $n$ by $\max(t - A[n], 0)$
  - at index $n+1$ by $\max(t - A[n+1], 0)$
  - ...
  - at index $2n-1$ by $\max(t - A[2n-1], 0)$.

**Algorithm (for decision problem)**

Iterate over indices $i$ from $n$ to $2n-1$ to calculate

$$\sum_{i=n}^{2n-1} \max(t - A[i], 0).$$

If this value is $\leq k$, report yes; otherwise report no.

- This algorithm clearly runs in $O(n)$ in the worst case.

# Example problem: Maximum Median



## Decision problem

Given positive integers $t$ and $k$. Ling wants to know whether she can make the median of her numbers at least the target value $t$ (by adding 1 to one of her numbers each day for $k$ days).

- **Solved!** We can now test whether it's possible to reach (or exceed) any target in $O(n)$ time.
- Can we use this to solve our original optimisation problem?

## Optimisation Problem

At the end of the $k$ days, what is the largest possible median $t$ that Ling's array can have?

- **Solution approach.** Apply our decision algorithm for different target values $t$. Which values of $t$ do we need to test?

# Example problem: Maximum Median



- The relevant range for the target value $t$ is $A[n], \ldots, A[n] + k$
- **Solution to optimisation problem.** Apply our decision algorithm to every target value $t$ from $A[n]$ through to $A[n] + k$.
- **Running time**. We test $k + 1$ different target values, each taking $O(n)$ time. This gives a time complexity of $O((k + 1)n) = O(kn)$.
- **Question.** Can we obtain a polynomial running time? Are there any target values in this range that we can skip?
  - If we test a target $t$, does the result give us any information about <u>other</u> targets?

# Example problem: Maximum Median



- **Question.** If we test a target $t$, does the result give us any information about <u>other</u> targets?
  - Monotonicity: targets are all achievable until some cutoff, and not achievable thereafter.
- **Better solution to optimisation problem.**
  - We can binary search on "is target $t$ achievable?" for $t$ in the range $[A[n], A[n] + k]$.
  - We don't have a pre-computed array of yes/no values; instead we'll compute them as they're needed in the binary search, by using our decision algorithm
- **Time complexity.** The binary search has $O(\log k)$ steps each taking $O(n)$ time, so the running time is $O(n \log k)$.

# Merge sort



- Steps:
  - **Divide:** Split the array into two equal parts ($\Theta(1)$)
  - **Conquer:** Sort each part recursively
  - **Combine:** Merge the two sorted subarrays ($\Theta(n)$)
- Recursion is $\log_2 n$ levels deep, with a total of $\Theta(n)$ time spent in each level.
- Time complexity is $\Theta(n \log n)$.

# Example: Counting the number of inversions



**Problem**

Suppose that you have $m$ users ranking the same set of $n$ movies. You want to determine for any two users $A$ and $B$ how similar their tastes are (for example, in order to make a recommender system).

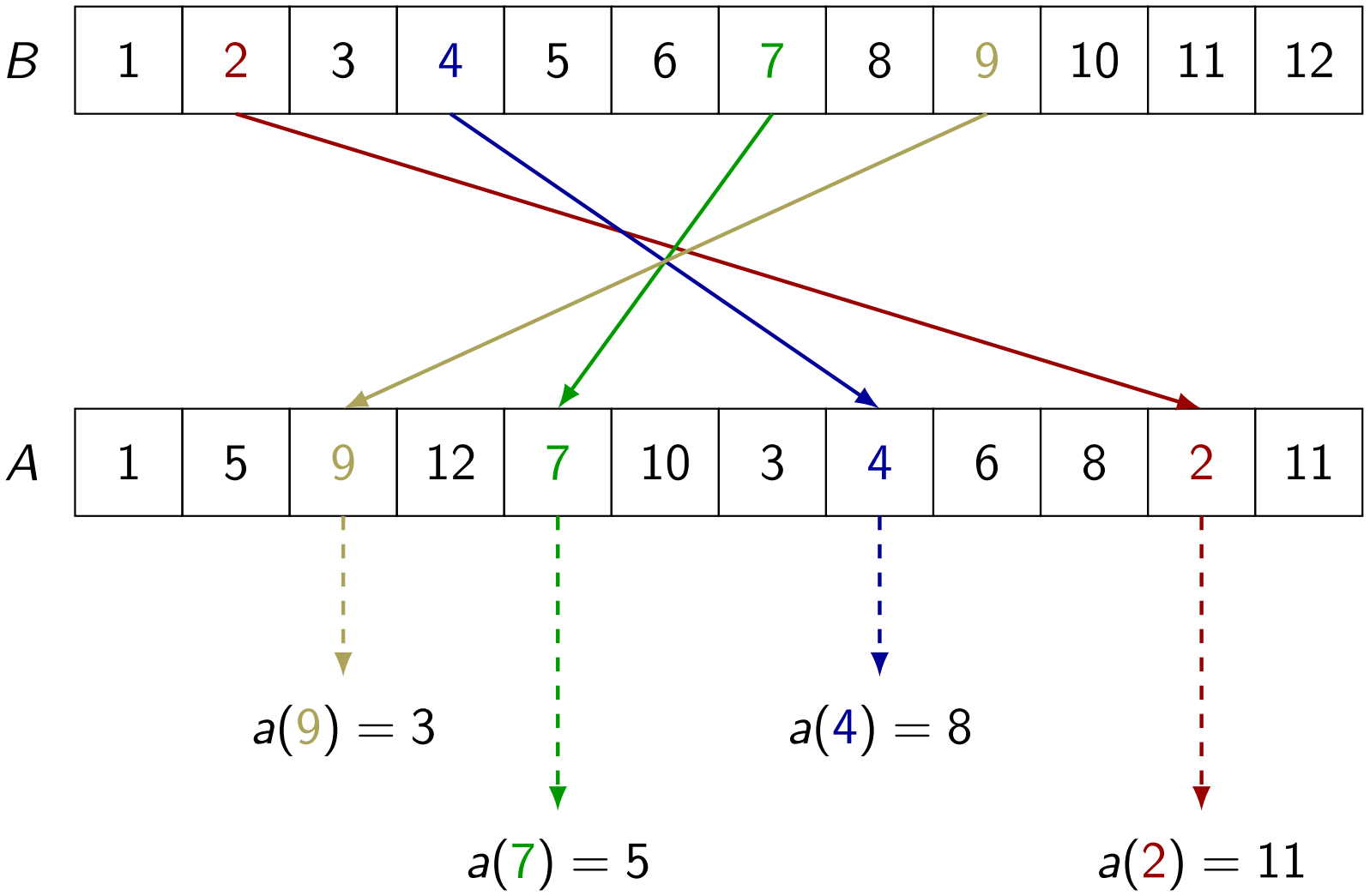
How should we measure the degree of similarity of two users $A$ and $B$?

**Approach:**

- Let's rank number the movies according to $B$'s ranking: assign index 1 to user $B$'s favourite movie, index 2 to their second favourite and so on.
- For each movie $i$, its rank in $A$'s list will be denoted by $a(i)$.
- Use the difference between each index $i$ (equal to $B$'s ranking), and value $a(i)$ (the ranking by $A$) to measure the difference between users $A$ and $B$.

# Example: Counting the number of inversions





# Example: Counting the number of inversions



**Question.** What's a good way to measure the difference between $A$ and $B$'s rankings?

There are lots of choices, some better than others.

The Kendall tau distance is the number of pairs of movies which are 'out of order' between the two lists.

**Definition**

An inversion is a pair $(i, j)$ such that:

- $i < j$, i.e., $B$ prefers $i$ to $j$, and
- $a(i) > a(j)$, i.e., $A$ prefers $j$ to $i$.

We will measure the difference between the users by counting the total number of inversions.

# Example: Counting the number of inversions



| | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $B$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
| $A$ | 1 | 5 | 9 | 12 | 7 | 10 | 3 | 4 | 6 | 8 | 2 | 11 |

For example, movies 1 and 2 do not form an inversion because both users prefer movie 1:

$$1 < 2 \text{ and } a(1) < a(2).$$

However, $B$ prefers movie 4 to 7 and $A$ disagrees, so this pair is an inversion:

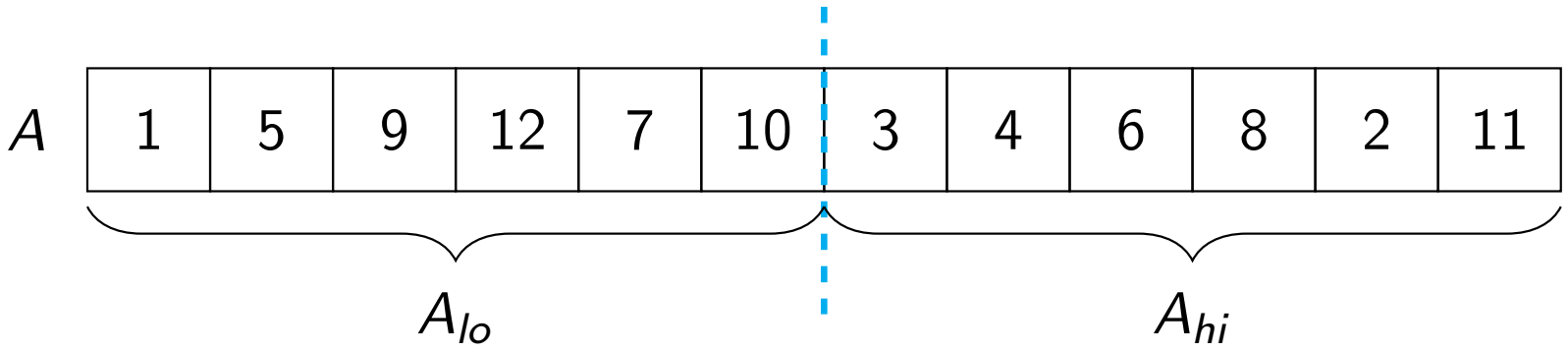
$$4 < 7 \text{ and } a(4) > a(7).$$

# Example: Counting the number of inversions



**Question.** How do we count the number of inversions?

- A brute force algorithm to count the inversions is to test each pair $i < j$, and add one to the total if $a(i) > a(j)$. This produces a quadratic time algorithm, $T(n) = \Theta(n^2)$.
- We now show that this can be done more efficiently, in time $O(n \log n)$, by applying a divide-and-conquer strategy.
- Clearly, since the total number of pairs is quadratic in $n$, we cannot afford to count the inversions one-by-one.
- The main idea is to tweak the merge sort algorithm, by extending it to recursively both sort an array $A$ **and** determine the number of inversions in $A$.




**Algorithm (adapting mergesort to count inversions):**

- We split the array $A$ into two (approximately) equal parts $A_{lo} = A[1..m]$ and $A_{hi} = A[m+1..n]$, where $m = \lfloor n/2 \rfloor$.
- Note that the total number of inversions in $A$ is the sum of:
  - the number of inversions $I(A_{lo})$ in $A_{lo}$ (such as 9 and 7),
  - the number of inversions $I(A_{hi})$ in $A_{hi}$ (such as 4 and 2), and
  - the number of inversions $I(A_{lo}, A_{hi})$ across the two halves (such as 7 and 4).

# Algorithm: Counting the number of inversions



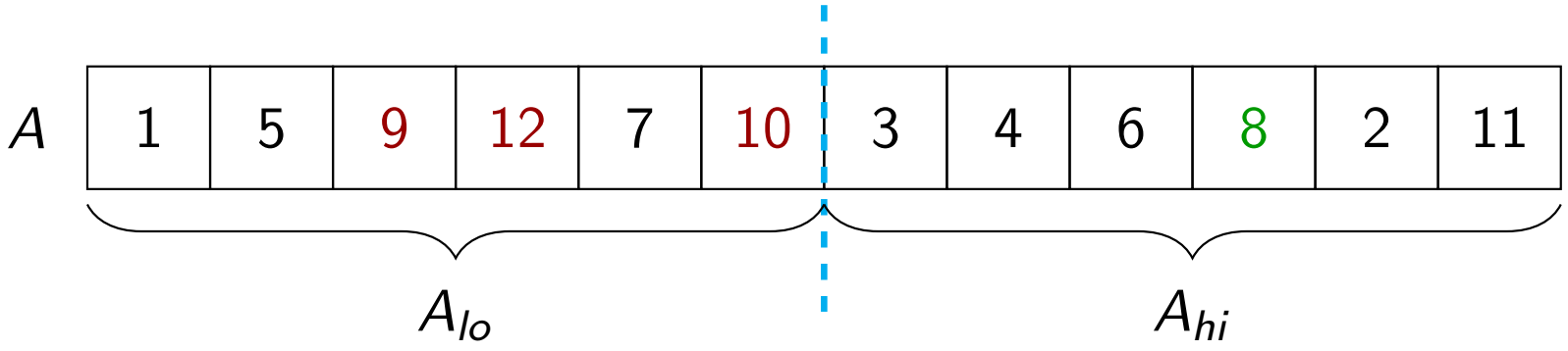
- We have

$$I(A) = I(A_{lo}) + I(A_{hi}) + I(A_{lo}, A_{hi}).$$

- The first two terms of the right-hand side are the number of inversions within $A_{lo}$ and within $A_{hi}$, which can be calculated recursively.

- The main challenge is to evaluate the last term, which requires us to count the inversions which cross the partition between the two sub-arrays.

- In our example, how many of these cross-partition inversions involve the 8?




- It's the number of elements of $A_{lo}$ which are greater than 8.
- How would one compute this systematically?

# Example: Counting the number of inversions



- The idea is to not only count inversions across the partition, but also sort the array. We can then assume that the subarrays $A_{lo}$ and $A_{hi}$ are sorted in the process of counting $I(A_{lo})$ and $I(A_{hi})$.

- We proceed to count $I(A_{lo}, A_{hi})$ (specifically, counting each inversion according to the smallest of its elements) and simultaneously merge as in merge sort.

In the merge step, each time we add an element of $A_{hi}$ to $A$, we have inversions between this number and each of the remaining elements in $A_{lo}$. We therefore add the number of elements remaining in $A_{lo}$ to the answer.

| | | | | | | |
|---|---|---|---|---|---|---|
| $A_{lo}$ | 1 | 5 | 7 | 9 | 10 | 12 |

| | | | | | | |
|---|---|---|---|---|---|---|
| $A_{hi}$ | 2 | 3 | 4 | 6 | 8 | 11 |

| | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $A$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | | | | |

$$\text{count} = 5 + 5 + 5 + 4 + 3 + 1 = 23.$$

# Algorithm: Counting the number of inversions



- On the other hand, when we add an element of $A_{lo}$ to $A$, all inversions involving this number have already been counted.

- We have therefore counted the number of inversions within each subarray ($I(A_{lo})$ and $I(A_{hi})$) recursively as well as the number of inversions across the partition ($I(A_{lo}, A_{hi})$), and adding these gives $I(A)$ as required.

- **Time complexity.** Our modified merge still takes linear time, so this algorithm has the same running time as merge sort, i.e., $\Theta(n \log n)$.

# Quicksort



- Steps:
  - **Divide:** Choose a pivot and partition the array around it ($\Theta(n)$)
  - **Conquer:** Sort both sides of the pivot recursively
  - **Combine:** Pass the answer up the recursion tree ($\Theta(1)$)
- Typically, recursion is $\log_2 n$ levels deep, with a total of $\Theta(n)$ time spent in each level.
- Time complexity is $\Theta(n \log n)$ in the average case.
- Worst case is $\Theta(n^2)$, with extreme choices of pivot.

# How do divide-and-conquer algorithms work?



- **Divide:** split up a large instance into smaller instances of the same type
- **Conquer:** apply the same algorithm to recursively solve each small instance, <u>indepedently of each other</u>
- **Combine:** synthesise the solution to the original instance from the solutions of the smaller instances
- Base cases are the smallest instances, where no further recursion is possible; usually straightforward.

# Correctness of divide-and-conquer algorithms



**Question.** How do we prove the correctness of divide-and-conquer algorithms?

- (Strong) induction on instance size.
- If the base cases are solved correctly, **and** the combine step is correct, then instances of all sizes are solved correctly.

## Example

In merge sort, the base cases are singleton subarrays, which are already sorted.
Assume that arrays of size up to $k$ are correctly sorted by merge sort. Then a subarray of size $k + 1$ is correctly sorted because:

- its two constituent subarrays are correctly solved, by the earlier assumption, and
- the merge procedure correctly combined them into the desired sorted array (with justification, if this is not self-evident).

# Table of Contents



1. Introductory Examples

2. Recurrences
2.1 Framework
2.2 Master Theorem

3. Integer Multiplication
3.1 Applying D&C to multiplication of large integers
3.2 The Karatsuba trick

4. Convolutions
4.1 Polynomials
4.2 The Fast Fourier Transform

5. Self-study: Sorting and selection
5.1 Quicksort and quickselect
5.2 Median of medians

6. Puzzle

# Table of Contents



1. Introductory Examples

2. Recurrences
2.1 Framework
2.2 Master Theorem

3. Integer Multiplication
3.1 Applying D&C to multiplication of large integers
3.2 The Karatsuba trick

4. Convolutions
4.1 Polynomials
4.2 The Fast Fourier Transform

5. Self-study: Sorting and selection
5.1 Quicksort and quickselect
5.2 Median of medians

6. Puzzle

- Recurrences are important to us because they arise in estimations of time complexity of divide-and-conquer algorithms.
- **Example.** Recurrence relation for counting inversions
  - Recall that counting inversions in an array $A$ of size $n$ required us to:
    - recurse on each half of the array ($A_{lo}$ and $A_{hi}$), and
    - count inversions across the partition, in linear time.
  - Denote the running time for an array of size $n$ by $T(n)$.
  - Then the running time $T(n)$ satisfies

$$T(n) = 2T\left(\frac{n}{2}\right) + c\,n$$

for some constant $c$.

**General recurrence relation for divide-and-conquer algorithms.**

- Let $a \geq 1$ be an integer and $b > 1$ be a real number, and suppose that a divide-and-conquer algorithm:
  - reduces an instance of size $n$ to $a$ instances of size $n/b$,
  - with overhead cost of $f(n)$ to split up the instance and combine the solutions from these smaller instances.
- Then the time complexity of such an algorithm satisfies

$$T(n) = a\,T\left(\frac{n}{b}\right) + f(n).$$

**Note**

Technically, we should be writing

$$T(n) = a\,T\left(\left\lceil \frac{n}{b} \right\rceil\right) + f(n)$$

but it can be shown that the same asymptotics are achieved if we ignore the rounding and additive constants.

# Solving Recurrences



- Some recurrences can be solved explicitly, but this tends to be tricky.
- Fortunately, we do not need the exact solution of a recurrence to discuss the efficiency of an algorithm.
- We only need to find:
  - the growth rate of the solution i.e., its asymptotic behaviour, and
  - where relevant, the (approximate) sizes of the constants involved.
- This is what the **Master Theorem** provides, when it is applicable.
- First, we will build some intuition.

# Table of Contents



1. Introductory Examples

2. Recurrences
2.1 Framework
2.2 Master Theorem

3. Integer Multiplication
3.1 Applying D&C to multiplication of large integers
3.2 The Karatsuba trick

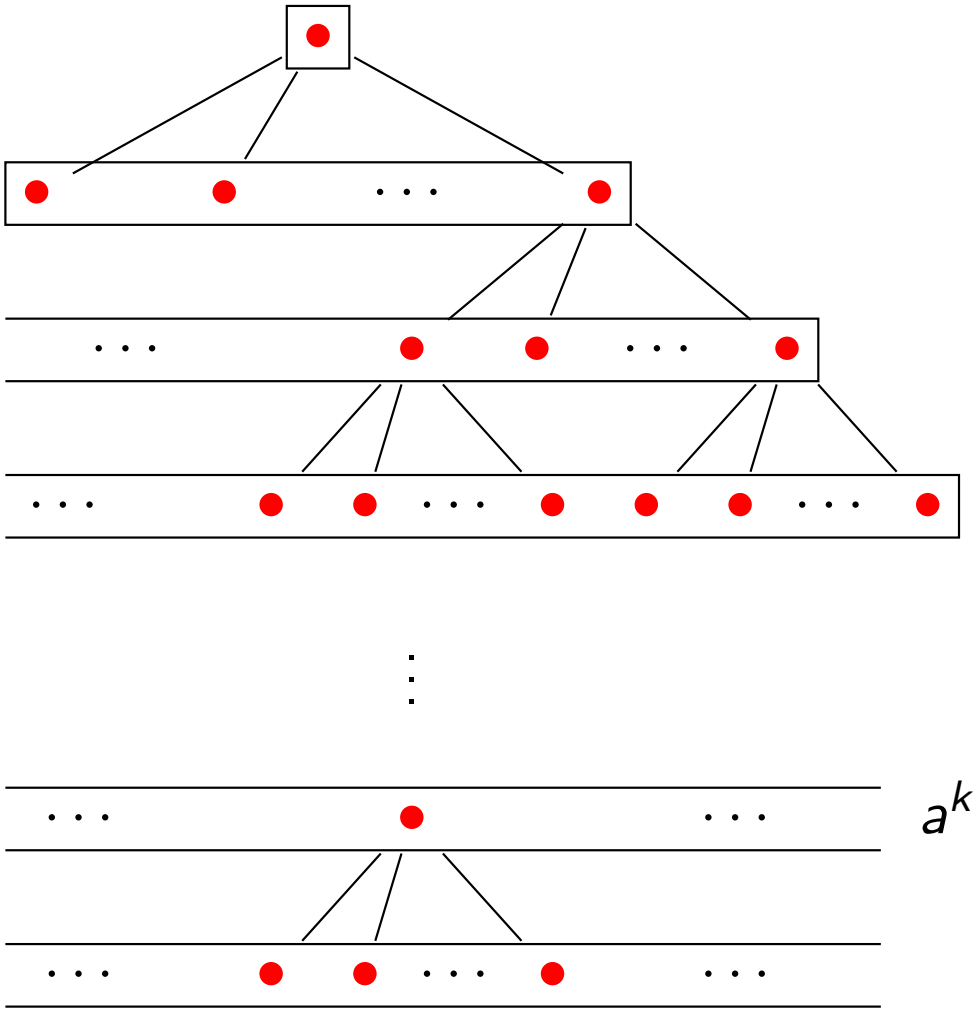
4. Convolutions
4.1 Polynomials
4.2 The Fast Fourier Transform

5. Self-study: Sorting and selection
5.1 Quicksort and quickselect
5.2 Median of medians

6. Puzzle

# Recurrences of the form $T(n) = a\,T\left(\frac{n}{b}\right) + f(n)$





1 instance of size $n$

$a$ instances of size $\frac{n}{b}$

$a^2$ instances of size $\frac{n}{b^2}$

$\vdots$ $\vdots$

$a^{k-1}$ instances of size $\frac{n}{b^{k-1}}$

$a^k$ instances of size $\frac{n}{b^k}$

**Running time to solve the base cases:**

- What is the depth of the tree?
  - Instances at level $k$ have size $n/b^k$.
  - This instance size $(n/b^k)$ becomes 1 when $k = \log_b n$.
- How many leaves does the tree have?
  - The leaves have instance size 1. We have seen this happens at depth $k = \log_b n$.
  - There are $a^k$ instances at depth $k$, so the number of leaves is $a^{\log_b n}$, which we can rewrite[1] as $n^{\log_b a}$.
- How much time is spent at the leaves?
  - Leaf nodes represent base cases, so each takes constant time for a total of $\Theta\left(n^{\log_b a}\right)$.

[1] To prove this, take $\log_b$ of both expressions and use the rule $\log_x y^z = z \log_x y$.

**How does the running time at the other levels compare?**

- As we go down the tree,
  - the number of instances increases by a factor of $a$ at each level, but
  - the size of those instances decreases by a factor of $b$ at each level.
- There are three main cases:
  1. If branching by a factor of $a$ outweighs the time saved by solving smaller instances, then nearly all the work is done in the leaves.
  2. If the effects of branching and smaller instances are about equivalent, then each level requires about the same amount of work.
  3. If branching is outweighed by the reduction in instance size, then nearly all the work is done at the root.

## Setup

Let:

- $a \geq 1$ be an integer and $b > 1$ be a real number;
- $f(n) > 0$ be a non-decreasing function defined on the positive integers;
- $T(n)$ be the solution of the recurrence

$$T(n) = a\,T(n/b) + f(n).$$

Define the <u>critical exponent</u> $c^* = \log_b a$ and the <u>critical polynomial</u> $n^{c^*}$. Observe that $n^{c^*} = n^{\log_b a}$ is just the number of leaves in the tree.

**Theorem**

1. If $f(n) = O(n^{c^* - \varepsilon})$ for some $\varepsilon > 0$, then $T(n) = \Theta(n^{c^*})$.
2. If $f(n) = \Theta(n^{c^*})$, then[2] we have $T(n) = \Theta(n^{c^*} \log n)$.
3. If $f(n) = \Omega(n^{c^* + \varepsilon})$ for some $\varepsilon > 0$, **and** for some $k < 1$ and some $n_0$, we have

$$a\, f\,(n/b) \leq k\, f(n) \tag{1}$$

holds for all $n > n_0$ (the <u>regularity condition</u>), **then** $T(n) = \Theta(f(n))$.

[2] There are extensions to this case, but we won't use them.

### Remark

- Recall that for $a, b > 1$, $\log_a n = \Theta(\log_b n)$, so we can omit the base and simply write statements of the form $f(n) = \Theta(g(n) \log n)$.
- However, $n^{\log_a x}$ is not interchangeable with $n^{\log_b x}$ - the base must be specified in such expressions.

### Exercise

Prove that $f(n) = \Omega(n^{c^* + \varepsilon})$ is a consequence of (1).

# Using the Master Theorem - Template



If you are asked to apply the Master Theorem to a given example recurrence relation

$$T(n) = aT(n/b) + f(n),$$

(where $a$ and $b$ are specified), your solution should work through the following steps:

- Calculate the critical exponent $c^* = \log_b a$.
- Calculate the critical polynomial $n^{c^*}$.
- Compare $f(n)$ to the critical polynomial to understand its asymptotic bounds: $O(), \Omega()$ or $\Theta()$.
- See if one of the rules of the Master Theorem can be applied, and if so, apply it!

Next are some examples of how to do this.

### Example 1

Let $T(n) = 4\,T(\frac{n}{2}) + n$.
Then the critical exponent is $c^* = \log_b a = \log_2 4 = 2$, so the critical polynomial is $n^2$.
Now, $f(n) = n = O(n^{2-\varepsilon})$ for suitable small $\varepsilon$.
In this case any $\varepsilon$ with $0 < \varepsilon < 1$ works, e.g. 0.1.
This satisfies the condition for case 1, so $T(n) = \Theta(n^2)$.

### Example 2

Let $T(n) = 2\,T(\frac{n}{2}) + 5\,n$.
Then the critical exponent is $c^* = \log_b a = \log_2 2 = 1$, so the critical polynomial is $n^{c^*} = n$.
Now, $f(n) = 5\,n = \Theta(n)$.
This satisfies the condition for case 2, so $T(n) = \Theta(n \log n)$.

### Example 3

Let $T(n) = 3\,T(\frac{n}{4}) + n$.
Then the critical exponent is $c^* = \log_4 3 \approx 0.7925$, so the critical polynomial is $n^{\log_4 3}$.
Now, $f(n) = n = \Omega(n^{\log_4 3+\varepsilon})$ for suitable small $\varepsilon$.
For example, $\varepsilon = 0.1$ is sufficient, because

$$\log_4 3 + \varepsilon \approx 0.7925 + 0.1 = 0.8925 < 1.$$

We also need to check the regularity condition!

### Example 3 (continued)

Recall that the recurrence gave $a = 3$, $b = 4$ and $f(n) = n$.
Case 3 of the Master Theorem is only applicable if for some $k < 1$ and some $n_0$,

$$a\,f\,(n/b) \leq k\,f(n)$$

holds for all $n > n_0$.
The left-hand side is $\frac{3n}{4}$, so if we choose any $k$ between $\frac{3}{4}$ and 1 then the required inequality will hold for all $n > n_0 = 0$.

### Example 3 (continued)

Therefore we can choose $k = 0.9$ and $n_0 = 0$, so that whenever $n > n_0$ we have

$$a\,f\left(\frac{n}{b}\right) = 3\,f\left(\frac{n}{4}\right) = \frac{3n}{4} < k\,n = k\,f(n).$$

This satisfies the conditions for case 3, so $T(n) = \Theta(f(n)) = \Theta(n)$.

## Limitations

Not every recurrence meets the conditions of the Master Theorem!

If the recurrence is of the form $T(n) = a\,T\left(\frac{n}{b}\right) + f(n)$, with the usual preconditions as stated on slide 42, then the theorem might not apply because:

- $f(n)$ fails the regularity condition for case 3,[3] or
- $f(n)$ falls between two of the cases.

We won't require you to solve these recurrences on your own.

[3] This is rare in the analysis of actual algorithms. The regularity condition must always be checked, but it usually only fails for contrived recurrences, e.g. $T(n) = 2T(\frac{n}{2}) + n\,(2 - \cos n)$.

### Example 4

Let $T(n) = 2\,T(\frac{n}{2}) + n\log_2(\log_2 n)$.
Then the critical exponent is $c^* = \log_2 2 = 1$, so the critical polynomial is $n$.
Now, $f(n) = n\log_2(\log_2 n) \neq O(n)$, so cases 1 and 2 do not apply.[4]
However, $f(n) \neq \Omega(n^{1+\varepsilon})$, no matter how small we choose $\varepsilon > 0$, so case 3 does not apply either.
Therefore the Master Theorem does **not** apply!

[4] Even the extensions of case 2 do not help here; they cover only the cases where $f(n) = \Theta(n^{c^*}\log^s n)$ for some integer $s$.

Suppose $T(n)$ satisfies the recurrence

$$T(n) = a\left[T\left(\frac{n}{b}\right)\right] + f(n) \tag{2}$$

However, the $T(n/b)$ term can itself be reduced using the recurrence as follows:

$$T\left(\frac{n}{b}\right) = a\,T\left(\frac{n}{b^2}\right) + f\left(\frac{n}{b}\right)$$

Substituting into (2) and simplifying gives

$$\begin{aligned} T(n) &= a\left[aT\left(\frac{n}{b^2}\right) + f\left(\frac{n}{b}\right)\right] + f(n) \\ &= a^2\,T\left(\frac{n}{b^2}\right) + a\,f\left(\frac{n}{b}\right) + f(n). \end{aligned}$$

We have now established

$$T(n) = a^2\left[T\left(\frac{n}{b^2}\right)\right] + a\,f\left(\frac{n}{b}\right) + f(n). \tag{3}$$

But why stop there? We can now reduce the $T\left(n/b^2\right)$ term, again using (2):

$$T\left(\frac{n}{b^2}\right) = a\,T\left(\frac{n}{b^3}\right) + f\left(\frac{n}{b^2}\right).$$

We now substitute this into (3) and simplify to get

$$\begin{aligned} T(n) &= a^2\left[a\,T\left(\frac{n}{b^3}\right) + f\left(\frac{n}{b^2}\right)\right] + a\,f\left(\frac{n}{b}\right) + f(n) \\ &= a^3\,T\left(\frac{n}{b^3}\right) + a^2\,f\left(\frac{n}{b^2}\right) + a\,f\left(\frac{n}{b}\right) + f(n). \end{aligned}$$

We can see a pattern emerging!

Continuing in this way, we find that

$$\begin{aligned} T(n) &= a^k\, T\left(\frac{n}{b^k}\right) + a^{k-1}\, f\left(\frac{n}{b^{k-1}}\right) + \ldots + a\, f\left(\frac{n}{b}\right) + f(n) \\ &= a^k\, T\left(\frac{n}{b^k}\right) + \sum_{i=0}^{k-1} a^i\, f\left(\frac{n}{b^i}\right). \end{aligned}$$

We stop when $k = \lfloor \log_b n \rfloor$, since this gives $n/b^k \approx 1$.

$$T(n) \approx a^{\log_b n}\, T\left(\frac{n}{b^{\log_b n}}\right) + \sum_{i=0}^{\lfloor \log_b n \rfloor - 1} a^i\, f\left(\frac{n}{b^i}\right).$$

Now we have

$$T(n) \approx a^{\log_b n}\, T\left(\frac{n}{b^{\log_b n}}\right) + \sum_{i=0}^{\lfloor \log_b n \rfloor - 1} a^i\, f\left(\frac{n}{b^i}\right).$$

We can use the identity $a^{\log_b n} = n^{\log_b a}$ to get:

$$T(n) \approx n^{\log_b a}\, T(1) + \underbrace{\sum_{i=0}^{\lfloor \log_b n \rfloor - 1} a^i\, f\left(\frac{n}{b^i}\right)}_{S}. \tag{4}$$

Importantly, we have not assumed anything about $f(n)$ yet! We will now analyse the sum $S$ in the simplest case of the Master Theorem, namely Case 2.

# Master Theorem proof: Case 2



Suppose $f(n) = \Theta\left(n^{\log_b a}\right)$. Then,

$$
\begin{aligned}
S &= \sum_{i=0}^{\lfloor \log_b n \rfloor - 1} a^i \, f\left(\frac{n}{b^i}\right) \\
&= \sum_{i=0}^{\lfloor \log_b n \rfloor - 1} a^i \, \Theta\left(\left(\frac{n}{b^i}\right)^{\log_b a}\right) \\
&= \Theta\left(\sum_{i=0}^{\lfloor \log_b n \rfloor - 1} a^i \, \left(\frac{n}{b^i}\right)^{\log_b a}\right) \\
&= \Theta\left(n^{\log_b a} \sum_{i=0}^{\lfloor \log_b n \rfloor - 1} a^i \, \left(\frac{1}{b^i}\right)^{\log_b a}\right)
\end{aligned}
$$

$$
\begin{aligned}
S &= \Theta\left(n^{\log_b a} \sum_{i=0}^{\lfloor \log_b n \rfloor - 1} \left(\frac{a}{b^{\log_b a}}\right)^i\right). \\
&= \Theta\left(n^{\log_b a} \sum_{i=0}^{\lfloor \log_b n \rfloor - 1} \left(\frac{a}{b^{\log_b a}}\right)^i\right) \\
&= \Theta\left(n^{\log_b a} \sum_{i=0}^{\lfloor \log_b n \rfloor - 1} \left(\frac{a}{a}\right)^i\right) \\
&= \Theta\left(n^{\log_b a} \sum_{i=0}^{\lfloor \log_b n \rfloor - 1} 1\right) \\
&= \Theta\left(n^{\log_b a} \lfloor \log_b n \rfloor\right).
\end{aligned}
$$

Finally, we return to (4). Substituting our result for $S$ gives

$$\begin{aligned} T(n) &\approx n^{\log_b a}\, T(1) + \Theta\left(n^{\log_b a} \lfloor \log_b n \rfloor\right) \\ &= \Theta\left(n^{\log_b a} \log n\right) \\ &\qquad \text{as logarithms of any base are equivalent,} \end{aligned}$$

completing the proof.

# Master Theorem proof: Other cases



- The proof of Case 1 is very similar to the above. The main difference is that $\sum 1$ is replaced by $\sum (b^{\varepsilon})^i$, forming a geometric series, which can be summed using the identity

$$1 + r + r^2 + \ldots + r^{k-1} = \frac{r^k - 1}{r - 1}.$$

- In Case 3, we need to prove that $T(n) = \Theta(f(n))$, that is, both:

  - $T(n) = \Omega(f(n))$, which follows directly from the recurrence $T(n) = a\,T(n/b) + f(n)$, and

  - $T(n) = O(f(n))$ (not as obvious).

### Exercise

Prove that in Case 3, $T(n) = O(f(n))$.

### Hint

You will need to bound

$$S = \sum_{i=0}^{\lfloor \log_b n \rfloor - 1} a^i f\left(\frac{n}{b^i}\right)$$

from above. Try using the inequality

$$a\, f\left(\frac{n}{b}\right) \leq c\, f(n)$$

to relate each term of $S$ to $f(n)$.

# Table of Contents



1. Introductory Examples

2. Recurrences
2.1 Framework
2.2 Master Theorem

3. Integer Multiplication
3.1 Applying D&C to multiplication of large integers
3.2 The Karatsuba trick

4. Convolutions
4.1 Polynomials
4.2 The Fast Fourier Transform

5. Self-study: Sorting and selection
5.1 Quicksort and quickselect
5.2 Median of medians

6. Puzzle

- In this section, we investigate how to multiply large integers quickly.
- An integer of value $N$ can be represented with $n = \log_2 N$ bits in base 2.
- Recall however that any other base is equivalent, since the difference between logarithms of different bases is a constant factor.

```
 1 0 0 0 1    <- carry
   1 0 1 0 1  <- first integer  (21)
+  1 1 0 0 1  <- second integer (25)
 -----------
 1 0 1 1 1 0  <- result         (46)
```

- Adding 3 bits can be done in constant time.
- It follows that the whole algorithm runs in linear time i.e., $O(n)$ many steps, where the two numbers are each $n$-bits long.

**Question**

Can we add two $n$-bit numbers in faster than in linear time?

**Answer**

No! There is no asymptotically faster algorithm because we have to use every bit of the input numbers, which takes $\Omega(n)$ time.

```
        1 1 0 1  <- first input integer  (13)
      * 1 0 1 0  <- second input integer (10)
        -------
        1 0 1 0  \
      0 0 0 0     \ O(n^2) intermediate operations:
    1 0 1 0       /   O(n^2) multiplications of bits
  1 0 1 0        /  + O(n^2) additions of bits
---------------
1 0 0 0 0 0 1 0  <- result of length 2n (130)
```

- Multiplying two bits is the same operation as logical AND, so clearly it takes constant time, but the same complexity applies for digits of other bases.
- Thus, the above procedure runs in time $O(n^2)$.

**Question**

Can we multiply two $n$-bit numbers in linear time, like addition?

**Answer**

**No one knows!** "Simple" problems can actually turn out to be difficult!

**Question**

Can we do it in faster than quadratic time? Let's try divide and conquer.

# Table of Contents



1. Introductory Examples

2. Recurrences
2.1 Framework
2.2 Master Theorem

3. Integer Multiplication
3.1 Applying D&C to multiplication of large integers
3.2 The Karatsuba trick

4. Convolutions
4.1 Polynomials
4.2 The Fast Fourier Transform

5. Self-study: Sorting and selection
5.1 Quicksort and quickselect
5.2 Median of medians

6. Puzzle

- Split the two input numbers $A$ and $B$ into halves:
  - $A_0, B_0$ - the least significant $n/2$ bits;
  - $A_1, B_1$ - the most significant $n/2$ bits.

$$A = A_1 2^{\frac{n}{2}} + A_0 \qquad \underbrace{XX\ldots X}_{\frac{n}{2}}\underbrace{XX\ldots X}_{\frac{n}{2}}$$

$$B = B_1 2^{\frac{n}{2}} + B_0$$

- $AB$ can now be calculated recursively using the following equation:

$$AB = A_1 B_1 2^n + (A_1 B_0 + B_1 A_0) 2^{\frac{n}{2}} + A_0 B_0.$$

### Algorithm

We recursively apply the following algorithm to multiply two integers $A$ and $B$, each consisting of $n$ bits.

The base case is $n = 1$, where we simply evaluate the product.

Otherwise, we let

- $A_1$ and $A_0$ be the most and least significant parts of $A$ respectively
- $B_1$ and $B_0$ be the most and least significant parts of $B$ respectively.

### Algorithm (continued)

Now, we compute

- $X = A_0 B_0$
- $Y = A_0 B_1$
- $Z = A_1 B_0$
- $W = A_1 B_1$

recursively, as each is a product of two $n/2$-bit integers.

Finally, we compute $W\,2^n + (Y + Z)\,2^{n/2} + X$ by summing

- $W$ shifted left by $n$ bits
- $Y$ and $Z$ each shifted left by $n/2$ bits, and
- $X$ with no shift.

# Multiplying large integers by divide-and-conquer



- How many steps does this algorithm take?
- Each multiplication of two $n$ digit numbers $(A, B)$ is replaced by four multiplications of $n/2$ digit numbers

$$X = A_0 B_0, Y = A_0 B_1, Z = A_1 B_0, W = A_1 B_1,$$

  plus we have a **linear** overhead to shift and add.
- Therefore the recurrence is

$$T(n) = 4\,T\left(\frac{n}{2}\right) + c\,n.$$

- Let's use the Master Theorem!

- From the recurrence

$$T(n) = 4\,T\left(\frac{n}{2}\right) + c\,n,$$

  we have $a = 4$, $b = 2$ and $f(n) = c\,n$.

- The critical exponent is $c^* = \log_2 4 = 2$, so the critical polynomial is $n^2$.

- Then $f(n) = c\,n = O(n^{2-0.1})$, so Case 1 applies.

- We conclude that $T(n) = \Theta(n^{c^*}) = \Theta(n^2)$, i.e., we gained **nothing** with our divide-and-conquer!

**Question**

Is there a smarter multiplication algorithm taking less than $O(n^2)$ many steps?

**Answer**

Remarkably, there is!

# Table of Contents



1. Introductory Examples

2. Recurrences
2.1 Framework
2.2 Master Theorem

3. Integer Multiplication
3.1 Applying D&C to multiplication of large integers
3.2 The Karatsuba trick

4. Convolutions
4.1 Polynomials
4.2 The Fast Fourier Transform

5. Self-study: Sorting and selection
5.1 Quicksort and quickselect
5.2 Median of medians

6. Puzzle

- In 1952, one of the most famous mathematicians of the $20^{th}$ century, Andrey Kolmogorov, conjectured that you cannot multiply in less than quadratic many elementary operations.

- In 1960, Anatoly Karatsuba, then a 23-year-old student, found an algorithm (later it was called "divide-and-conquer") that multiplies two $n$-digit numbers in $\Theta\left(n^{\log_2 3}\right) \approx \Theta(n^{1.58\ldots})$ elementary steps, thus disproving the conjecture!! Kolmogorov was shocked!

- Once again we split each of our two input numbers $A$ and $B$ into halves:

$$A = A_1 2^{\frac{n}{2}} + A_0 \qquad \underbrace{XX\ldots X}_{\frac{n}{2}}\underbrace{XX\ldots X}_{\frac{n}{2}}.$$

$$B = B_1 2^{\frac{n}{2}} + B_0$$

- Previously we saw that

$$AB = A_1 B_1 2^n + (A_1 B_0 + A_0 B_1) 2^{\frac{n}{2}} + A_0 B_0,$$

but rearranging the bracketed expression gives

$$AB = A_1 B_1 2^n + ((A_1 + A_0)(B_1 + B_0) - A_1 B_1 - A_0 B_0) 2^{\frac{n}{2}} + A_0 B_0,$$

saving one multiplication at each round of the recursion!

### Algorithm

We recursively apply the following algorithm to multiply two integers $A$ and $B$, each consisting of $n$ bits.

The base case is $n = 1$, where we simply evaluate the product.

Otherwise, we let

- $A_1$ and $A_0$ be the most and least significant parts of $A$ respectively
- $B_1$ and $B_0$ be the most and least significant parts of $A$ respectively.

## Algorithm (continued)

Now, we compute

- $X = A_0 B_0$
- $W = A_1 B_1$
- $V = (A_1 + A_0)(B_1 + B_0)$

recursively, as each is a product of two (approximately) $n/2$-bit integers.

Finally, we compute $W\,2^n + (V - W - X)\,2^{n/2} + X$ by summing

- $W$ shifted left by $n$ bits
- $V$ less $W$ and $X$ shifted left by $n/2$ bits, and
- $X$ with no shift.

# The Karatsuba trick



- How many steps does this algorithm take?
- Each multiplication of two $n$ digit numbers is replaced by three multiplications of $n/2$ digit numbers

$$X = A_0 B_0, W = A_1 B_1, V = (A_1 + A_0)(B_1 + B_0)$$

  with linear additional work to shift and add.

- Therefore, the recurrence is

$$T(n) = 3\,T\left(\frac{n}{2}\right) + c\,n.$$

- From the recurrence

$$T(n) = 3T\left(\frac{n}{2}\right) + c\,n,$$

we have $a = 3$, $b = 2$ and $f(n) = c\,n$.

- Now the critical exponent is $c^* = \log_2 3$. Once again, we are in Case 1 of the Master Theorem, but this time

$$\begin{aligned} T(n) &= \Theta\left(n^{\log_2 3}\right) \\ &= \Theta\left(n^{1.58\ldots}\right) \\ &\neq \Omega(n^2), \end{aligned}$$

disproving Kolmogorov's conjecture.

- We can do even better, by splitting the input numbers $A$ and $B$ into more than two pieces.
- In fact, for any $\epsilon > 0$, we can achieve $T(n) = O\left(n^{1+\epsilon}\right)$.
- However, with numbers divided into $p+1$ pieces, the constant factors involved are on the order of $p^p$, so it is only worthwhile for extremely large inputs.
- Moral: in practice, we can only rely on asymptotic estimates if the constants hidden by the big-oh notation are known to be reasonably small!

**Breakthrough**

Harvey and van der Hoeven (2021) gave an $O(n \log n)$ algorithm,[5] although it may only be faster than previous methods for astronomically large numbers.

[5] You can find the paper here, and media coverage and FAQ here.

- If we split the inputs into $p$ pieces, the problem is then to multiply

$$A = A_{p-1}2^{(p-1)k} + \ldots + A_2 2^{2k} + A_1 2^k + A_0$$

and

$$B = B_{p-1}2^{(p-1)k} + \ldots + B_2 2^{2k} + B_1 2^k + B_0,$$

where $k = \frac{n}{p}$ is the length of each piece.

- The key observation is that $2^{2k}, \ldots, 2^{(p-1)k}$ are the powers of $2^k$, but there is otherwise nothing special about powers of 2.
- So we can think of $A$ and $B$ as polynomials in $x$ evaluated at $x = 2^k$.

# Extensions of the Karatsuba trick



- We could instead work on the more general problem of multiplying <u>polynomials</u>

$$P_A(x) = A_{p-1}x^{p-1} + \ldots + A_2x^2 + A_1x + A_0$$

and

$$P_B(x) = B_{p-1}x^{p-1} + \ldots + B_2x^2 + B_1x + B_0.$$

- The objective is to compute the coefficients of $P_A(x)P_B(x)$ using as few multiplications as possible.
- It turns out this is a very important practical problem.
- Not limited to integer coefficients.

# Table of Contents



1. Introductory Examples

2. Recurrences
2.1 Framework
2.2 Master Theorem

3. Integer Multiplication
3.1 Applying D&C to multiplication of large integers
3.2 The Karatsuba trick

4. Convolutions
4.1 Polynomials
4.2 The Fast Fourier Transform

5. Self-study: Sorting and selection
5.1 Quicksort and quickselect
5.2 Median of medians

6. Puzzle

# Table of Contents



1. Introductory Examples

2. Recurrences
2.1 Framework
2.2 Master Theorem

3. Integer Multiplication
3.1 Applying D&C to multiplication of large integers
3.2 The Karatsuba trick

4. Convolutions
4.1 Polynomials
4.2 The Fast Fourier Transform

5. Self-study: Sorting and selection
5.1 Quicksort and quickselect
5.2 Median of medians

6. Puzzle

### Definition

A polynomial is a function of the form

$$P(x) = A_n x^n + A_{n-1} x^{n-1} + \ldots + A_1 x + A_0.$$

The $A_i$ are called coefficients, $n$ is the degree, and $x$ is a formal variable. Coefficients can be any kind of number, not just integers.

### Note

In this section, we assume that numbers can be added and multiplied in constant time. We'll see that this often involves real and even complex number arithmetic, so in practice it would be approximated using floating-point numbers, but if each coefficient fits into one word, the exact results of all intermediary computations can be represented with at most 3 words.

# Polynomial operations



- **Addition:** given two polynomials $P_A$ and $P_B$, construct the sum $P_A + P_B$ (which is itself a polynomial).
- **Evaluation:** given a polynomial $P$ and an input $x_0$, find the value of $P(x_0)$.
- **Multiplication:** given two polynomials $P_A$ and $P_B$, construct the product $P_A P_B$ (which is itself a polynomial).

# Coefficient representation



- The coefficient representation

$$P(x) = A_n x^n + A_{n-1} x^{n-1} + \ldots + A_1 x + A_0$$

  allows addition term-by-term in $O(n)$ time.
- Evaluation appears complicated because of the powers. Apply Horner's rule:
  - Other than $A_0$, all terms have at least one factor of $x$, so

$$P(x) = A_0 + x(A_1 + A_2 x + \ldots + A_{n-1} x^{n-2} + A_n x^{n-1}).$$

  - Now the same situation applies within the brackets! Continuing thus,

$$P(x) = A_0 + x(A_1 + x(A_2 + x(\ldots))).$$

  - Evaluate from the interior: start with $A_n$, multiply by $x$, add $A_{n-1}$, multiply by $x$, and so on. $O(n)$ time.

# Coefficient representation



- The main weakness of the coefficient representation is multiplication, which takes quadratic time.

- Distributing the product

$$(A_n x^n + \ldots + A_1 x + A_0)(B_n x^n + \ldots + B_1 x + B_0)$$

  involves multiplying every pair $A_i B_j$.

- Let's aim for a faster running time for this important problem!

- The real significance of polynomial multiplication comes from analysing the coefficient sequence of the product.
- If the input polynomials are

$$P_A(x) = A_n x^n + A_{n-1} x^{n-1} + \ldots + A_1 x + A_0$$

and

$$P_B(x) = B_n x^n + B_{n-1} x^{n-1} + \ldots + B_1 x + B_0,$$

the product will be of degree $2n$, say

$$P_C(x) = C_{2n} x^{2n} + C_{2n-1} x^{2n-1} + \ldots + C_1 x + C_0.$$

- When we expand the product $P_A(x)\, P_B(x)$, the product will have many terms with the same power of $x$, which are collected to form a term of the product $P_C(x)$.

- For example, distributing the brackets gives

$$
\begin{aligned}
&(\ldots + A_2x^2 + A_1x + A_0)(\ldots + B_2x^2 + B_1x + B_0) \\
&= \ldots + \left(A_2x^2B_0 + A_1xB_1x + A_0B_2x^2\right) + \ldots \\
&= \underbrace{\ldots}_{\text{terms in } x^{2n},\ldots,x^4,x^3} + \left(A_2B_0 + A_1B_1 + A_0B_2\right)x^2 + \underbrace{\ldots}_{\text{terms in } x^1,x^0}.
\end{aligned}
$$

- Therefore, equating coefficients of $x^2$ gives

$$C_2 = A_2B_0 + A_1B_1 + A_0B_2 = \sum_{i+j=2} A_iB_j,$$

reflecting all the ways that $P_A(x)$ can contribute an $x^i$ term and $P_B(x)$ an $x^j$ term that together make up $x^2$.

# Convolution



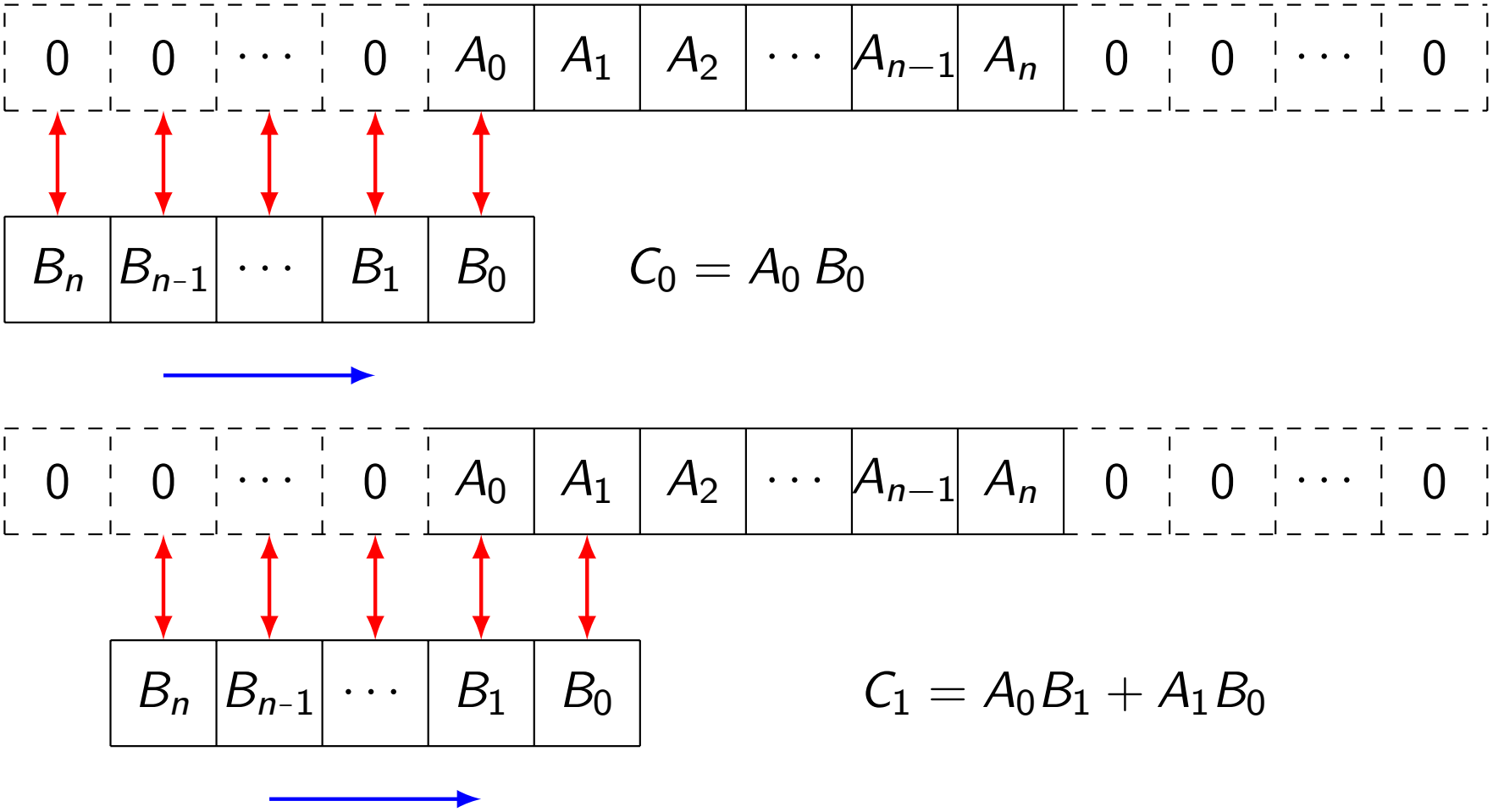
- In general, each coefficient of $P_C(x)$ is given by

$$C_t = \sum_{i+j=t} A_i B_j.$$

- We call this the <u>convolution</u> of the sequences $A$ and $B$, denoted $A \star B$.
- So far, we have only seen how to directly compute the product of two polynomials, which is equivalent to working out each term of the convolution using the formula above, as both involve multiplying each $A_i$ by each $B_j$.
- This takes $\Theta(n^2)$ time, as there are quadratic many products $A_i B_j$ in total.

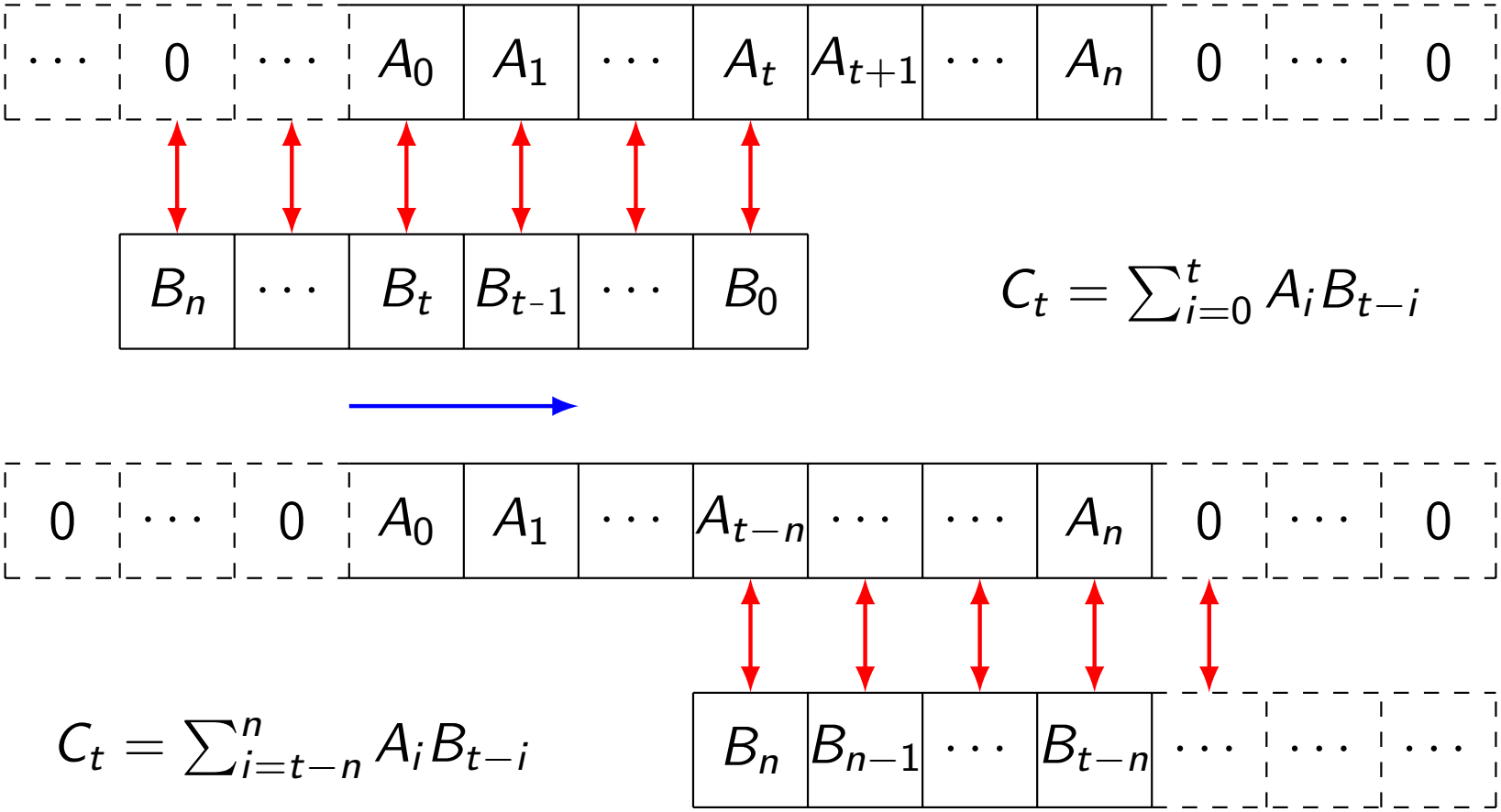
- In the remainder of this module, we'll see how to compute the convolution <u>indirectly</u> in $\Theta(n \log n)$ time.
  - We'll convert to a different representation in $\Theta(n \log n)$ using a divide-and-conquer algorithm, compute the product (easily, in $\Theta(n)$) and convert back using a very similar algorithm.
- $n$ here represents the degree of the input polynomials. If the polynomials have different degree, we'll use the larger one (which is equivalent to zero-padding the smaller one to the same length).

# Visualising convolution $C = A \star B$



# Visualising convolution $C = A \star B$






$C_t = \sum_{i=0}^{t} A_i B_{t-i}$

$C_t = \sum_{i=t-n}^{n} A_i B_{t-i}$

In general, $C_t = \displaystyle\sum_{i=\max(t-n,0)}^{\min(t,n)} A_i B_{t-i}$

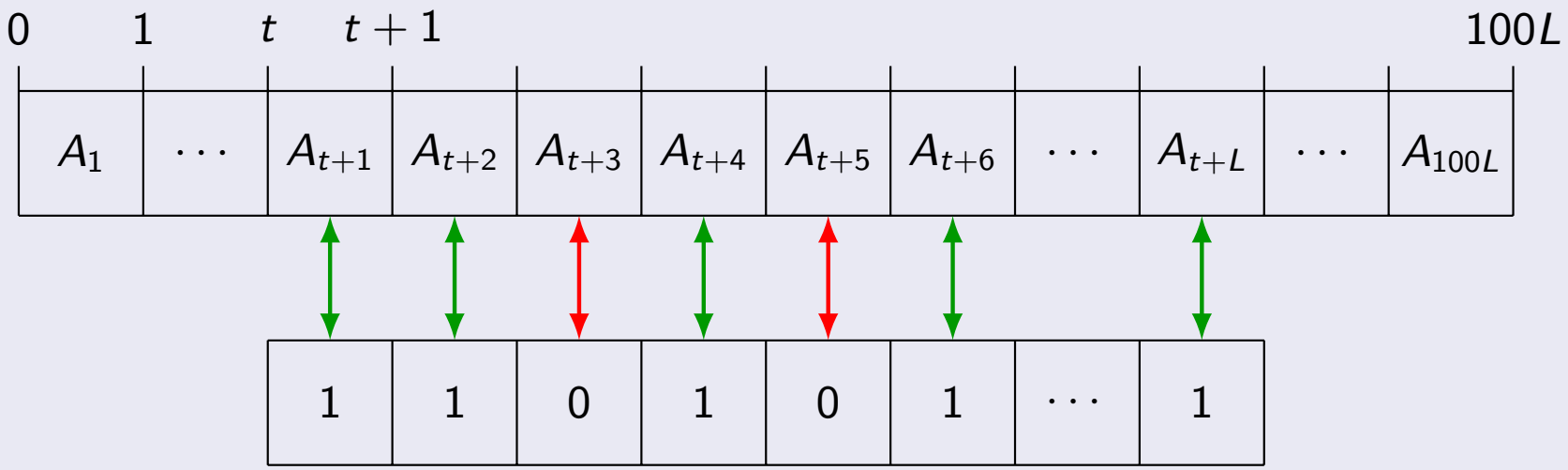
## Problem

Suppose you are given a map of a straight sea shore of length $100L$ metres, represented as a sequence of $100L$ integers where $A_i$ with $1 \leq i \leq 100L$ is the number of fish between the $(i-1)^{\text{th}}$ metre of the shore and the $i^{\text{th}}$ metre.

You also have a net of length $L$ metres, but, unfortunately, it has holes in it. The net is described as a sequence $N = (N_1, N_2, \ldots, N_L)$ of $L$ ones and zeros, where 0's denote where the holes are.

If you throw the net starting at metre $t$ and ending at metre $t + L$, then you will catch only the fish in one metre stretches of the shore where the corresponding bit of the net is 1.

## Problem (continued)




$$C = A_{t+1} + A_{t+2} + A_{t+4} + A_{t+6} + \ldots + A_{t+L}$$

Find the spot where you should place the left end of your net in order to catch the largest possible number of fish using an algorithm which runs in time $O(L \log L)$.

# Example problem: Fishing Net



- If we position the left edge of the fishing net against the left edge of the sea shore, we will catch:
  - $A_1$ fish in the first metre if $N_1 = 1$, or zero otherwise,
  - $A_2$ fish in the next metre if $N_2 = 1$, or zero otherwise,
  - ...
  - $A_L$ fish in the last metre if $N_L = 1$, or zero otherwise,

  for a total of

$$A_1 N_1 + A_2 N_2 + \ldots A_L N_L = \sum_{i=1}^{L} A_i N_i.$$

- Similarly, the number of fish that can be caught from metre $t$ to metre $t + L$ is

$$\sum_{i=1}^{L} A_{t+i} N_i = \sum_{i=t+1}^{t+L} A_i N_{i-t}.$$

# Example problem: Fishing Net



- This expression

$$\sum_{i=t+1}^{t+L} A_i N_{i-t}$$

  looks a lot like a term of the convolution $A \star N$, but the indexing isn't quite right.

- We want the $A$-index to increase as the $N$-index <u>decreases</u> (or vice versa).

- We were instead hoping to see

$$\sum_{i=\ldots}^{\ldots} A_i N_{t-i}$$

.

## Solution

Let $N'$ be the net sequence $N$ in reverse order.

Then the number of fish that can be caught from metre $t - L$ to metre $t$ is

$$\sum_{i=t-L+1}^{t} A_i N'_{t-i}.$$

This now corresponds to the $t^{\text{th}}$ term of the convolution $A \star N'$ if we pad by 0's as necessary.

Therefore, we compute $A \star N'$ in $\Theta(L \log L)$ time[6] using the Fast Fourier Transform[7] and report the largest term of that sequence.

[6] Both sequences each of length at most $100L$, and as usual we can ignore the constant factor 100 in the asymptotic analysis.

[7] This is the divide-and-conquer algorithm that we develop in the rest of this module.

# Value representation



- Is there an alternative representation of polynomials that could be more useful for multiplication?
- We can instead represent a polynomial of degree $n$ by a sequence of $n + 1$ values!
  - There is a unique line through any two points in the plane, a unique parabola through any three points, and so on.
- Fix a selection of inputs $x_0, \ldots, x_n$. Then the corresponding values $P(x_0), \ldots, P(x_n)$ represent a unique polynomial of degree up to $n$, just as much as the coefficient representation would.

- Addition and multiplication can be executed <u>pointwise</u> in linear time:

$$(P_A + P_B)(x_0) = P_A(x_0) + P_B(x_0)$$

and

$$(P_A P_B)(x_0) = P_A(x_0) P_B(x_0).$$

- For multiplication, we'll need enough sampling points to uniquely determine the product polynomial $(A_n x^n + \ldots)(B_n x^n + \ldots)$.

- But there isn't a straightforward way to evaluate at an input which isn't among the $x_i$.

# Our strategy to multiply polynomials fast



1. Evaluate (quickly, somehow) $P_A(x)$ and $P_B(x)$ at $2n+1$ distinct points
$$x_0, x_1, \ldots, x_{2n}$$
2. Multiply them point by point using $2n+1$ multiplications of large numbers, to get $2n+1$ values of the product polynomial $P_C(x)$
3. Reconstruct (quickly, somehow) the coefficients of $P_C(x)$, i.e.
$$P_C(x) = C_{2n}x^{2n} + C_{2n-1}x^{2n-1} + \ldots + C_1 x + C_0.$$

# Our strategy to multiply polynomials fast



$$
\begin{array}{ccccc}
P_A(x) & \xrightarrow{\text{evaluate}} & P_A(x_0) & \cdots & P_A(x_{2n}) \\
 & & \times & & \times \\
P_B(x) & \xrightarrow{\text{evaluate}} & P_B(x_0) & \cdots & P_B(x_{2n}) \\
 & & \shortparallel & & \shortparallel \\
P_C(x) & \xleftarrow[\text{reconstruct}]{} & P_C(x_0) & \cdots & P_C(x_{2n})
\end{array}
$$

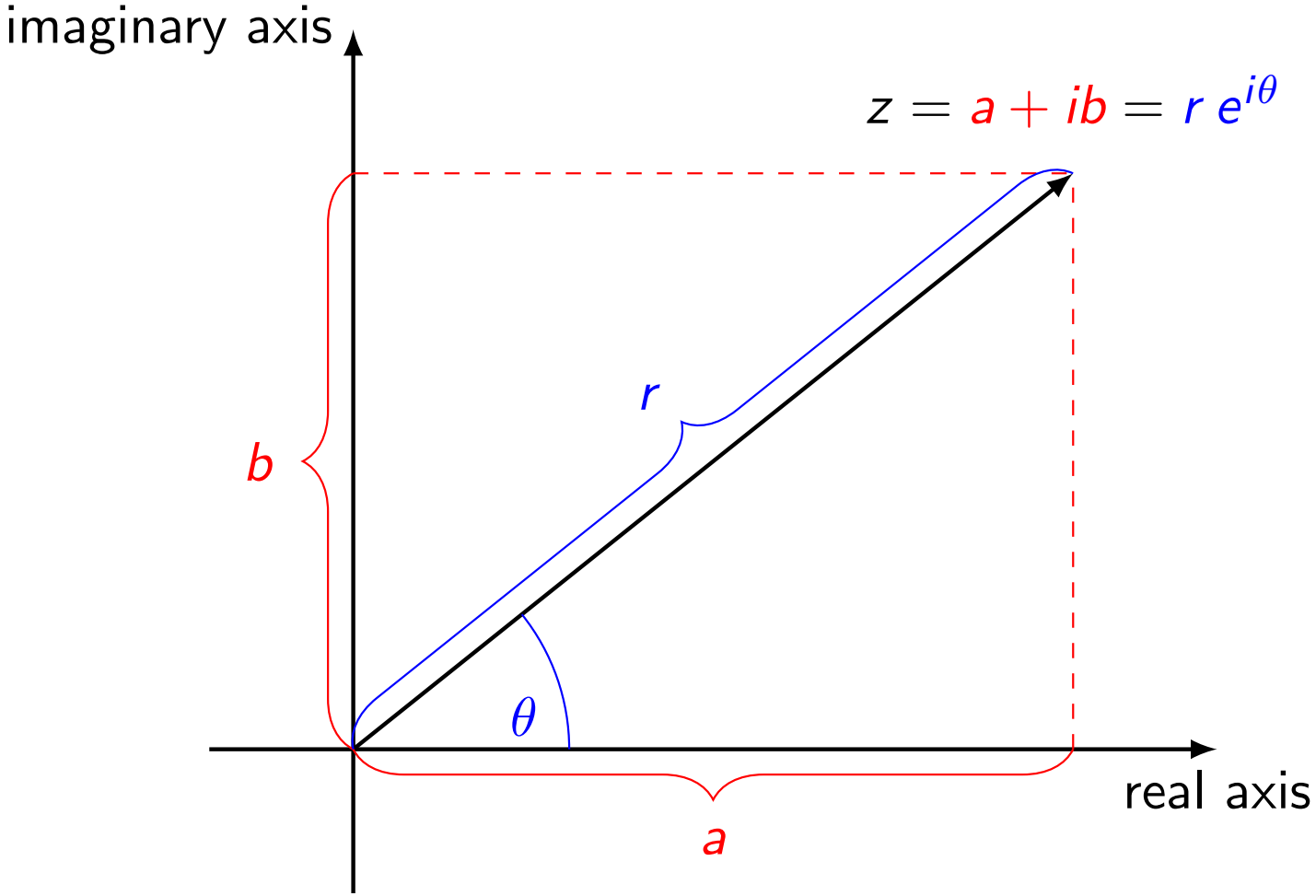
# Our strategy to multiply polynomials fast



- What inputs $x_0, \ldots, x_{2n}$ should we choose?
- $-n, \ldots, 0, \ldots, n$ seems like a natural choice
  - Pro: we may be able to re-use some computation between positive-negative pairs $a$ and $-a$.
  - Con: this re-use may work for one level of recursion, but a serious attempt at this reveals it is hard to make work for further levels, since $a^2 = (-a)^2$.
  - Con: polynomials take values of the order of $n^n$. While ok in the unit-cost model of computation, this creates issues in more realistic models of computation, such as the word-RAM model, where we assume only unit-cost operations on $\log n$-sized words.
- Complex numbers to the rescue!

- Let $z = a + ib$ be a complex number, where $a, b \in \mathbb{R}$ and $i = \sqrt{-1}$.
- Then we can define the
  - <u>modulus</u> $|z|$, a positive real number $r$ such that $r = \sqrt{a^2 + b^2}$ and
  - <u>argument</u> $\arg z$, an angle $\theta \in (-\pi, \pi]$ such that $a = r\cos\theta$ and $b = r\sin\theta$.
- Then we have

$$\begin{aligned} z &= a + ib \\ &= r(\cos\theta + i\sin\theta) \\ &= r\,e^{i\theta}. \qquad \text{(Euler's formula)} \end{aligned}$$


imaginary axis
$z = a + ib = r\,e^{i\theta}$
$r$
$b$
$\theta$
real axis
$a$

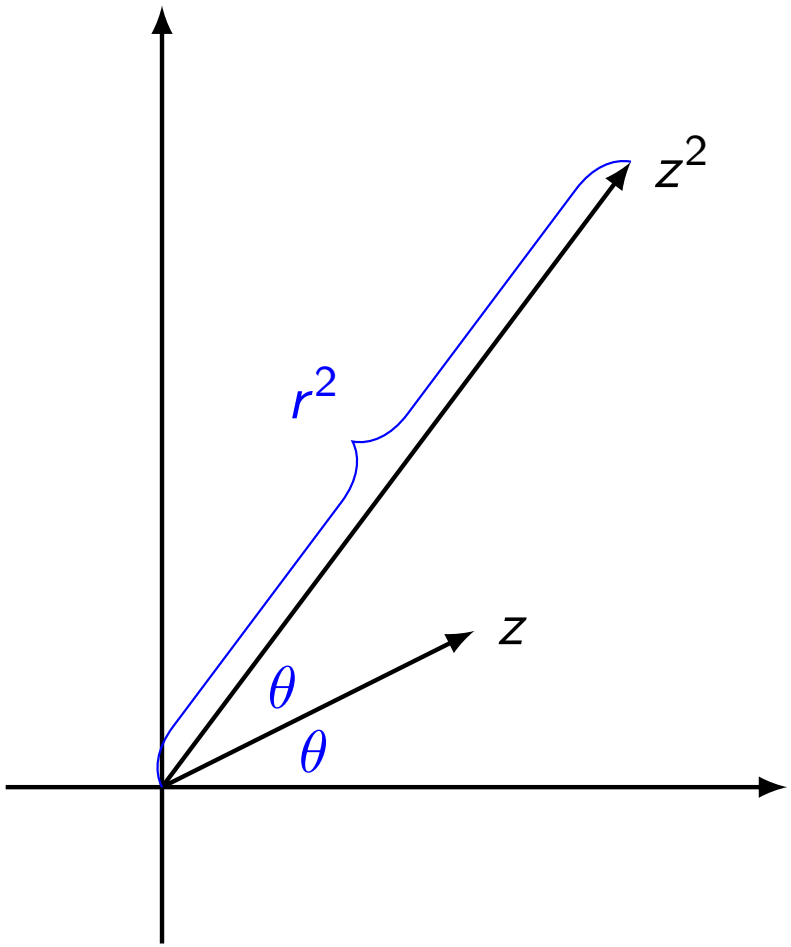
To multiply two complex numbers, we multiply the moduli and add the arguments.

**Theorem**

If $z = r\,e^{i\theta}$ and $w = s\,e^{i\phi}$, then $z\,w = (r\,s)\,e^{i\,(\theta+\phi)}$.

**Corollary**

In particular, if $z = r\,e^{i\theta}$ then $z^n = r^n\,e^{i\,(n\theta)}$.


$z^2$
$r^2$
$z$
$\theta$
$\theta$

# Complex roots of unity



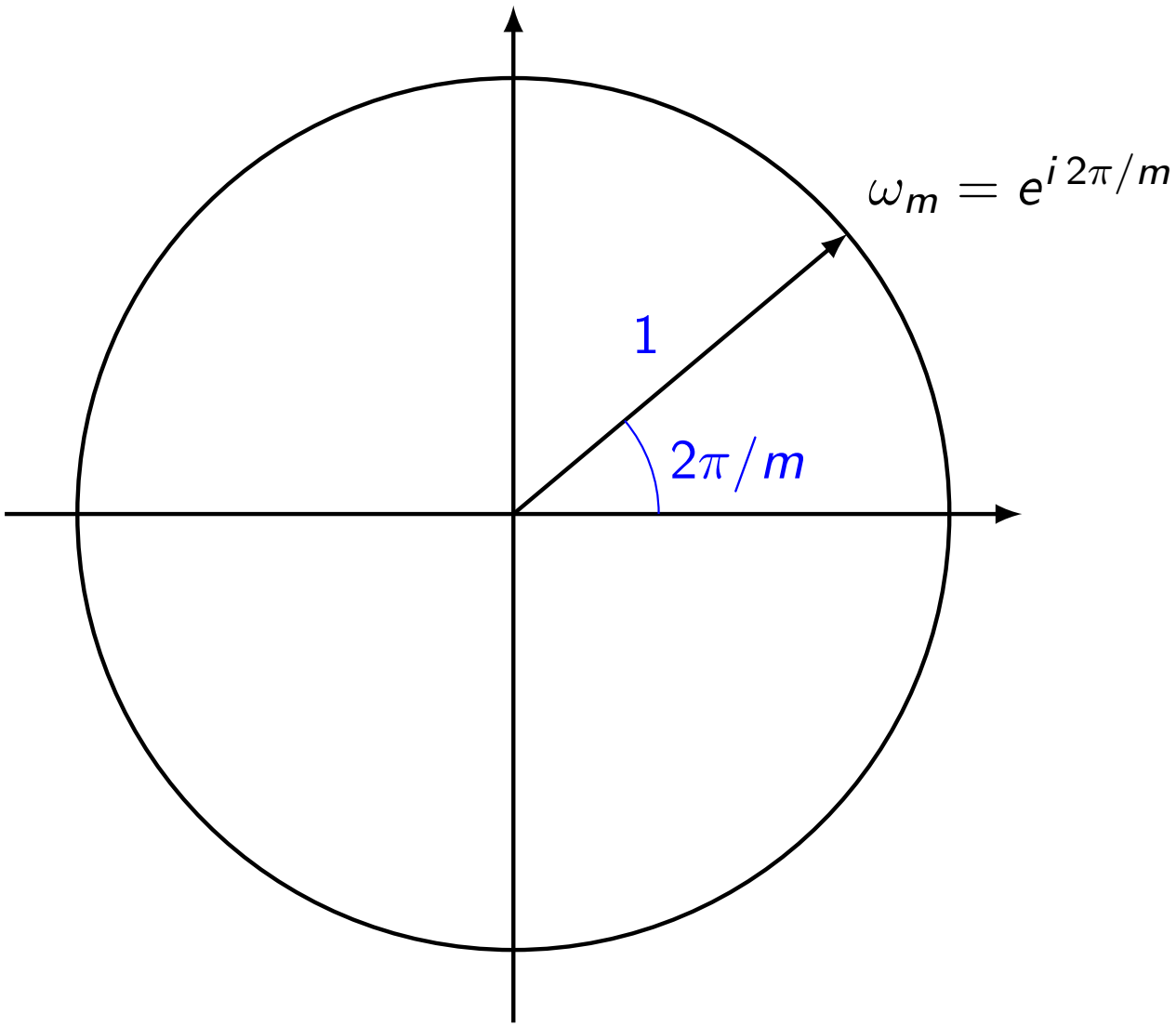
- We are interested in the solutions of $z^m = 1$, known as <u>roots of unity of order</u> $m$.
- Solving $r^m\, e^{i\,(m\theta)} = 1$, we see that

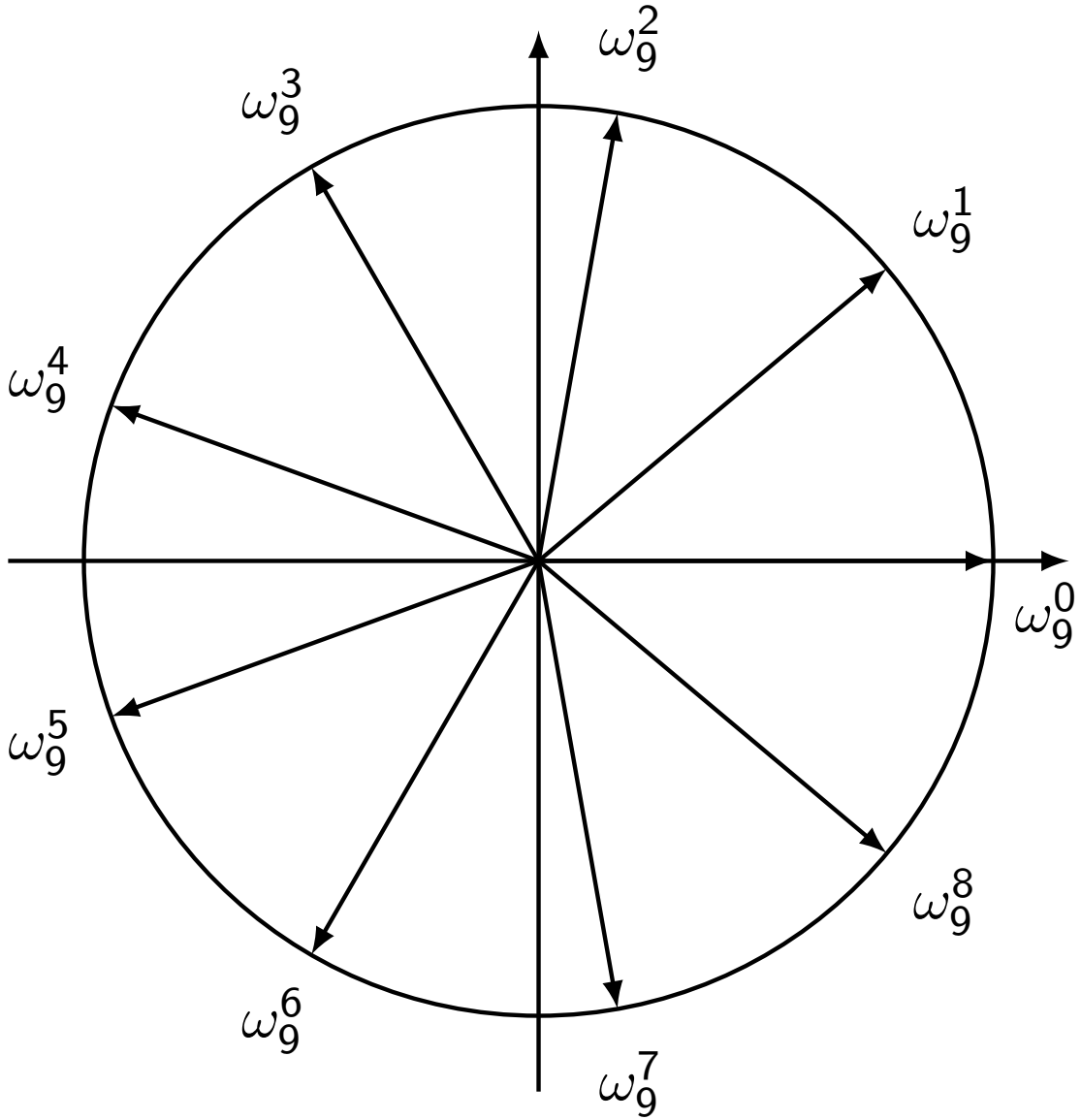
$$r^m = 1, \text{ i.e., } r = 1$$

and

$$m\theta = 2\pi\, k, \text{ i.e., } \theta = \frac{2\pi\, k}{m} \qquad \text{for some } k \in \mathbb{Z}.$$

- Let $\omega_m = e^{i\,2\pi/m}$. Then $z = \omega_m^k$, i.e., all roots of unity of order $m$ can be written as powers of $\omega_m$. We say that $\omega_m$ is a <u>primitive root of unity of order</u> $m$.
- Note that there are only $m$ distinct values here, as $\omega_m^m = \omega_m^0 = 1$. So we only need to consider integer values of $k$ such that $0 \leq k \leq m-1$.


$\omega_m = e^{i\,2\pi/m}$
1
$2\pi/m$


$\omega_9^2$
$\omega_9^3$
$\omega_9^1$
$\omega_9^4$
$\omega_9^0$
$\omega_9^5$
$\omega_9^8$
$\omega_9^6$
$\omega_9^7$

- For $\omega_m = e^{i\,2\pi/m}$ and for all integers $k$ such that $0 \leq k \leq m-1$,

$$((\omega_m)^k)^m = (\omega_m)^{m\,k} = ((\omega_m)^m)^k = 1^k = 1.$$

- Thus, $\omega_m^k = (\omega_m)^k$ is also a root of unity.

**Theorem**

$\omega_m^k$ is a **primitive** root of unity of order $m$ if and only if $\gcd(m, k) = 1$.

- The product of two roots of unity ($\omega_m^i$ and $\omega_m^j$) of order $m$ is given by
$$\omega_m^i \omega_m^j = \omega_m^{i+j},$$
which is itself a root of unity of the same order.
- So the set of all roots of unity of order $m$, i.e., $\{1, \omega_m, \omega_m^2, \ldots, \omega_m^{m-1}\}$ is closed under multiplication (and by extension, under taking powers).
- Note that this is not true for addition, i.e., the sum of two roots of unity is NOT another root of unity!

**Note**

This is an example of a group.

### Cancellation Lemma

For all positive integers $\ell, m$ and integers $k$:

$$\omega_{\ell m}^{\ell k} = \omega_m^k.$$

### Proof

$$\omega_{\ell m}^{\ell k} = \left(e^{i\frac{2\pi}{\ell m}}\right)^{\ell k} = e^{i\frac{2\pi \ell k}{\ell m}} = e^{i\frac{2\pi k}{m}} = \left(e^{i\frac{2\pi}{m}}\right)^k = \omega_m^k.$$

- In particular, $(\omega_{2m}^k)^2 = \omega_{2m}^{2k} = (\omega_{2m}^2)^k = \omega_m^k$, i.e. the squares of the roots of unity of order $2m$ are just the roots of unity of order $m$.

# Table of Contents



1. Introductory Examples

2. Recurrences
2.1 Framework
2.2 Master Theorem

3. Integer Multiplication
3.1 Applying D&C to multiplication of large integers
3.2 The Karatsuba trick

4. Convolutions
4.1 Polynomials
4.2 The Fast Fourier Transform

5. Self-study: Sorting and selection
5.1 Quicksort and quickselect
5.2 Median of medians

6. Puzzle

# Our strategy to multiply polynomials fast



- Given two polynomials of degree at most $n$,

$$P_A(x) = A_n x^n + \ldots + A_0 \text{ and } P_B(x) = B_n x^n + \ldots + B_0,$$

we want to find $P_C(x) = P_A(x)\, P_B(x)$.

1. Evaluate $P_A(x)$ and $P_B(x)$ at $m = 2n + 1$ distinct points

$$\omega_m^0, \omega_m^1, \ldots, \omega_m^{m-1}$$

2. Multiply them point by point using $2n + 1$ multiplications of large numbers, to get $2n + 1$ values of $P_C(x)$

3. Reconstruct the coefficients of $P_C(x)$, i.e.

$$P_C(x) = C_{2n} x^{2n} + C_{2n-1} x^{2n-1} + \ldots + C_1 x + C_0.$$

$$
\begin{array}{ccccc}
P_A(x) & \xrightarrow{\text{evaluate}} & P_A\left(\omega_m^0\right) & \cdots & P_A\left(\omega_m^{m-1}\right) \\
 & & \times & & \times \\
P_B(x) & \xrightarrow{\text{evaluate}} & P_B\left(\omega_m^0\right) & \cdots & P_B\left(\omega_m^{m-1}\right) \\
 & & \| & & \| \\
P_C(x) & \xleftarrow[\text{reconstruct}]{} & P_C\left(\omega_m^0\right) & \cdots & P_C\left(\omega_m^{m-1}\right)
\end{array}
$$

## Definition

Let $A = \langle A_0, A_1, \ldots, A_{m-1} \rangle$ be a sequence of $m$ real or complex numbers, and let the corresponding polynomial be

$$P_A(x) = \sum_{j=0}^{m-1} A_j x^j.$$

Let $\widehat{A}_k = P_A(\omega_m^k)$ for all $0 \leq k \leq m-1$. The values of $P_A$ evaluated at the roots of unity of order $m$.

The sequence of values $\widehat{A} = \langle \widehat{A}_0, \widehat{A}_1, \ldots, \widehat{A}_{m-1} \rangle$ is called the <u>Discrete Fourier Transform</u> (DFT) of $A$.

The DFT is just the value representation of $P_A(x)$ at the $m$th roots of unity.

- Our polynomials $P_A(x)$ and $P_B(x)$ have degree $n$, so the corresponding sequences of coefficients ($A$ and $B$) each have only $n+1$ terms.
- Our goal polynomial $P_C(x)$ has degree $2n$, so we need to be able to calculate all $2n+1$ coefficients of (sequence $C$).
- However, we defined the DFT of a sequence as having the same length as the original sequence ($A$ and $B$). So to calculate the output sequence ($C$) we must obtain the values at all $m$ roots of unity of order $m = 2n+1$.
- This is easily rectified by padding the sequences $A$ and $B$ with zeros at the end, corresponding to the terms $0x^{n+1} + 0x^{n+2} + \ldots + 0x^{2n}$, since $m = 2n+1$.
- The DFT $\widehat{A}$ of a sequence $A$ can be computed <u>very fast</u> using a divide-and-conquer algorithm called the Fast Fourier Transform.

# Our strategy to multiply polynomials fast



- We can now compute the DFTs of the two (0 padded) sequences:

$$\mathrm{DFT}(\langle A_0, A_1, \ldots, A_n, \underbrace{0, \ldots, 0}_{n}\rangle) = \langle \widehat{A}_0, \widehat{A}_1, \ldots, \widehat{A}_{m-1}\rangle$$

  and similarly for $B$.
- For each $k$ we multiply the corresponding values $\widehat{A}_k = P_A(\omega_m^k)$ and $\widehat{B}_k = P_B(\omega_m^k)$, thus obtaining

$$\widehat{C}_k = \widehat{A}_k \widehat{B}_k = P_A(\omega_m^k) P_B(\omega_m^k) = P_C(\omega_m^k)$$

- We then use the inverse transformation for DFT, called IDFT, to recover the coefficients $\langle C_0, C_1, \ldots, C_{m-1}\rangle$ of the product polynomial $P_C(x)$ from the sequence $\langle \widehat{C}_0, \widehat{C}_1, \ldots, \widehat{C}_{m-1}\rangle$ of its values $C_k = P_C(\omega_m^k)$ at the roots of unity of order $m$.

$$\langle A_0, A_1, \ldots, A_n, \underbrace{0, \ldots, 0}_{n} \rangle \xrightarrow{\text{DFT}} \widehat{A}_0 \quad \cdots \quad \widehat{A}_{m-1}$$

$$\times \qquad\qquad \times$$

$$\langle B_0, B_1, \ldots, B_n, \underbrace{0, \ldots, 0}_{n} \rangle \xrightarrow{\text{DFT}} \widehat{B}_0 \quad \cdots \quad \widehat{B}_{m-1}$$

$$\| \qquad\qquad \|$$

$$\langle C_0, C_1, \ldots, C_{m-1} \rangle \xleftarrow[\text{IDFT}]{} \widehat{C}_0 \quad \cdots \quad \widehat{C}_{m-1}$$

### Question

How long does it take to compute the DFT of a sequence $A$, i.e. find the values

$$P_A(\omega_m^k) = A_0 + A_1\,\omega_m^k + A_2\,(\omega_m^k)^2 + \ldots + A_{m-1}\,(\omega_m^k)^{m-1}$$

for all $k$ such that $0 \leq k < m$?

### Answer

Even if we were to precompute all powers of $\omega_m$ (recall, there are only $m$ distinct values), there are $m$ multiplications required for each of $m$ values $P_A(\omega_m^k)$.

However, we can use divide-and-conquer to find all the required values in $O(n \log n)$ time!

- We can assume that $m$ is a power of 2 - otherwise we can pad $P_A(x)$ with zero coefficients until its number of coefficients becomes equal to the nearest power of 2.

**Exercise**

Show that for every $m$ which is not a power of two the smallest power of 2 larger or equal to $m$ is smaller than $2m$.

**Hint**

Consider $m$ in binary. How many bits does the nearest power of two have?

## Problem

Given a sequence $A = \langle A_0, A_1, \ldots, A_{m-1} \rangle$, compute its DFT, i.e., find the values $P_A(x)$ at $x = \omega_m^k$ for all $k$ such that $0 \leq k < m$.

## The main idea

We use divide-and-conquer by splitting the polynomial $P_A(x)$ into the even powers and the odd powers:

$$\begin{aligned} P_A(x) &= (A_0 + A_2\, x^2 + A_4\, x^4 + \ldots + A_{m-2}\, x^{m-2}) \\ &\quad + (A_1\, x + A_3\, x^3 + \ldots + A_{m-1}\, x^{m-1}) \\ &= \left(A_0 + A_2\, x^2 + A_4\, (x^2)^2 + \ldots + A_{m-2}\, (x^2)^{m/2-1}\right) \\ &\quad + x\left(A_1 + A_3\, x^2 + A_5\, (x^2)^2 + \ldots + A_{m-1}\, (x^2)^{m/2-1}\right) \end{aligned}$$

- Let us define $A^{[0]} = \langle A_0, A_2, A_4, \ldots A_{m-2} \rangle$ and $A^{[1]} = \langle A_1, A_3, A_5, \ldots A_{m-1} \rangle$, so that

$$P_{A^{[0]}}(y) = A_0 + A_2\, y + A_4\, y^2 + \ldots + A_{m-2}\, y^{m/2-1}$$
$$P_{A^{[1]}}(y) = A_1 + A_3\, y + A_5\, y^2 + \ldots + A_{m-1}\, y^{m/2-1}$$

  and hence

$$P_A(x) = P_{A^{[0]}}(x^2) + x\, P_{A^{[1]}}(x^2).$$

- Note that the polynomials $P_{A^{[0]}}(y)$ and $P_{A^{[1]}}(y)$ have $m/2$ coefficients each, while the polynomial $P_A(x)$ has $m$ coefficients.

**Question**

Have we successfully reduced a problem of size $m$ to a problem of size $m/2$?

- **Problem of size $m$:**
  Evaluate a polynomial with $m$ coefficients at all roots of unity of order $m$.

- **Problem of size $m/2$:**
  Evaluate a polynomial with $m/2$ coefficients at all roots of unity of order $m/2$.

- The original problem was to evaluate the polynomial $P_A(x)$ with $m$ coefficients at inputs $x = \omega_m^k$ for each $0 \leq k < m$. We have reduced it to the evaluation of two polynomials $P_{A^{[0]}}(y)$ and $P_{A^{[1]}}(y)$ each with $m/2$ coefficients at points

$$y = x^2 = \omega_m^{2k}.$$

- Recall that we assumed $m$ is a power of 2, and hence even. We can therefore use the cancellation lemma

$$y = \omega_m^{2k} = \omega_{m/2}^k$$

  to deduce that $y$ is a root of unity of order $m/2$.
- Note that $k$ goes up to $m-1$, but there are only $m/2$ distinct roots of unity of order $m/2$. Since $\omega_{m/2}^{m/2} = 1$, we can easily simplify the later terms as repetitions of the earlier terms.

- Therefore we can write

$$\begin{aligned} P_A(\omega_m^k) &= P_{A^{[0]}}\left((\omega_m^k)^2\right) + \omega_m^k \cdot P_{A^{[1]}}\left((\omega_m^k)^2\right) \\ &= P_{A^{[0]}}\left(\omega_{m/2}^k\right) + \omega_m^k \cdot P_{A^{[1]}}\left(\omega_{m/2}^k\right). \end{aligned}$$

- Thus, we have reduced a problem of size $m$ to two such problems of size $m/2$, plus a linear overhead to multiply the $\omega_m^k$ terms.

- We have recursively reduced evaluation of a polynomial $P_A(x)$ with $m$ coefficients at $m$ roots of unity of order $m$ to evaluations of two polynomials $P_{A^{[0]}}(y)$ and $P_{A^{[1]}}(y)$, each with $m/2$ coefficients, at $m/2$ many roots of unity of order $m/2$.

- Once we get these $m/2$ values of $P_{A^{[0]}}(y)$ and $P_{A^{[1]}}(y)$ we need $m$ additional multiplications to obtain the values of

$$\underbrace{P_A\left(\omega_m^k\right)}_{y_k} = \underbrace{P_{A^{[0]}}\left(\omega_{m/2}^k\right)}_{y_k^{[0]}} + \omega_m^k \underbrace{P_{A^{[1]}}\left(\omega_{m/2}^k\right)}_{y_k^{[1]}}$$

for all $0 \leq k < m$.

# How fast is the Fast Fourier Transform?



- Thus, we have reduced a problem of size $m$ to two such problems of size $m/2$, plus a linear overhead.

- Consequently, our algorithm's run time satisfies the recurrence

$$T(m) = 2\,T\left(\frac{m}{2}\right) + c\,m.$$

- Case 2 of the Master Theorem gives

$$T(m) = \Theta(m \log m) = \Theta(n \log n).$$

- The evaluation of a polynomial $P_A(x) = A_0 + A_1 x + \ldots + A_{m-1} x^{m-1}$ at roots of unity $\omega_m^k$ of order $m$ can be represented using the following matrix-vector product.

$$\underbrace{\begin{pmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega_m & \omega_m^2 & \cdots & \omega_m^{m-1} \\ 1 & \omega_m^2 & \omega_m^{2\cdot 2} & \cdots & \omega_m^{2\cdot(m-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega_m^{m-1} & \omega_m^{2(m-1)} & \cdots & \omega_m^{(m-1)(m-1)} \end{pmatrix}}_{M} \begin{pmatrix} A_0 \\ A_1 \\ A_2 \\ \vdots \\ A_{m-1} \end{pmatrix} = \begin{pmatrix} P_A(1) \\ P_A(\omega_m) \\ P_A(\omega_m^2) \\ \vdots \\ P_A(\omega_m^{m-1}) \end{pmatrix}$$

- The FFT is just a method of replacing this matrix-vector multiplication taking $m^2$ many multiplications with a $\Theta(m \log m)$ procedure.

- We also need a way to 'reconstruct' the coefficients from these values, i.e. perform the inverse DFT.

- If $M$ is invertible, we could write:

$$\begin{pmatrix} A_0 \\ A_1 \\ A_2 \\ \vdots \\ A_{m-1} \end{pmatrix} = M^{-1} \begin{pmatrix} P_A(1) \\ P_A(\omega_m) \\ P_A(\omega_m^2) \\ \vdots \\ P_A(\omega_m^{m-1}) \end{pmatrix}.$$

- Is $M$ invertible? How do we find $M^{-1}$?

**Claim**

The inverse of matrix $M$ is found by simply changing the signs of the exponents and dividing by $m$.

Let

$$Q = \frac{1}{m}\begin{pmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega_m^{-1} & \omega_m^{-2} & \dots & \omega_m^{-(m-1)} \\ 1 & \omega_m^{-2} & \omega_m^{-2\cdot 2} & \dots & \omega_m^{-2\cdot(m-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega_m^{-(m-1)} & \omega_m^{-2(m-1)} & \dots & \omega_m^{-(m-1)(m-1)} \end{pmatrix}.$$

We claim $Q$ is the inverse of $M$.
To prove this, we need to show $MQ = I (= QM)$, where $I$ is the identity matrix.

When we multiply $MQ$, the $(i, j)$ entry in the product matrix is given by the product of the $i$th row of $M$ and the $j$th column of $Q$:

$$(MQ)_{i,j} = \frac{1}{m}\begin{pmatrix} 1 & \omega_m^i & \omega_m^{2\cdot i} & \cdots & \omega_m^{i\cdot(m-1)} \end{pmatrix}\begin{pmatrix} 1 \\ \omega_m^{-j} \\ \omega_m^{-2j} \\ \vdots \\ \omega_m^{-(m-1)j} \end{pmatrix}$$

$$= \frac{1}{m}\sum_{k=0}^{m-1} \omega_m^{ik}\,\omega_m^{-jk} = \frac{1}{m}\sum_{k=0}^{m-1} \omega_m^{(i-j)k}$$

We have established that $(MQ)_{i,j} = \frac{1}{m}\sum_{k=0}^{m-1}\omega_m^{(i-j)k}$. We now have two possibilities:

1. if $i = j$, then

$$\begin{aligned}(MQ)_{i,j} &= \frac{1}{m}\sum_{k=0}^{m-1}\omega_m^0 \\ &= \frac{1}{m}\sum_{k=0}^{m-1}1 \\ &= \frac{1}{m}\cdot m = 1;\end{aligned}$$

2. if $i \neq j$, then $\sum_{k=0}^{m-1} \omega_m^{(i-j)k}$ represents the sum of a geometric progression with $m$ terms and common ratio $\omega_m^{i-j}$. Recalling that

$$1 + r + r^2 + \ldots + r^{m-1} = \frac{1 - r^m}{1 - r},$$

we have

$$\begin{aligned}
(MQ)_{i,j} &= \frac{1}{m} \cdot \frac{1 - \omega_m^{(i-j)m}}{1 - \omega_m^{i-j}} \\
&= \frac{1}{m} \cdot \frac{1 - (\omega_m^m)^{i-j}}{1 - \omega_m^{i-j}} \\
&= \frac{1}{m} \cdot \frac{1 - 1}{1 - \omega_m^{i-j}} = 0.
\end{aligned}$$

We have now proven that

$$(MQ)_{i,j} = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{otherwise} \end{cases},$$

that is, $MQ = I$. Therefore $Q$ is the inverse of $M$, i.e.

$$M^{-1} = \frac{1}{m} \begin{pmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega_m^{-1} & \omega_m^{-2} & \dots & \omega_m^{-(m-1)} \\ 1 & \omega_m^{-2} & \omega_m^{-2\cdot 2} & \dots & \omega_m^{-2\cdot(m-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega_m^{-(m-1)} & \omega_m^{-2(m-1)} & \dots & \omega_m^{-(m-1)(m-1)} \end{pmatrix}.$$

# Another remarkable feature of the roots of unity



So we get

$$\begin{pmatrix} A_0 \\ A_1 \\ A_2 \\ \vdots \\ A_{m-1} \end{pmatrix} = \frac{1}{m} \begin{pmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega_m^{-1} & \omega_m^{-2} & \cdots & \omega_m^{-(m-1)} \\ 1 & \omega_m^{-2} & \omega_m^{-2\cdot 2} & \cdots & \omega_m^{-2\cdot(m-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega_m^{-(m-1)} & \omega_m^{-2(m-1)} & \cdots & \omega_m^{-(m-1)(m-1)} \end{pmatrix} \begin{pmatrix} P_A(1) \\ P_A(\omega_m) \\ P_A(\omega_m^2) \\ \vdots \\ P_A(\omega_m^{m-1}) \end{pmatrix}.$$

This is not so different to the original matrix-vector product!

The 'reconstruct' step requires us to convert from the sequence of values

$$\langle P_A(1), P_A(\omega_m), P_A(\omega_m^2), \ldots, P_A(\omega_m^{m-1})\rangle,$$

which we denoted by

$$\langle \widehat{A}_0, \widehat{A}_1, \widehat{A}_2, \ldots, \widehat{A}_{m-1}\rangle$$

back to the sequence of coefficients

$$\langle A_0, A_1, A_2, \ldots, A_{m-1}\rangle,$$

i.e. to compute the inverse DFT.
We can use the same FFT algorithm with just two changes:

1. the root of unity $\omega_m$ is replaced by $\overline{\omega_m} = e^{-i\frac{2\pi}{m}}$,
2. the resulting output values are divided by $m$.

# Notation convention



- Computer science books take the forward DFT operation to be the evaluation of the corresponding polynomial at all roots of unity $\omega_m^k = \exp(2\pi k\, i/m)$ and the inverse DFT to be the evaluation of the polynomial at the complex conjugates of the roots of unity, i.e. at $\omega_m^{-k} = \exp(-2\pi k\, i/m)$.

- However, electrical engineering books do the opposite, with the direct DFT evaluating the polynomial at $\omega_m^{-k}$ and the inverse DFT at $\omega_m^k$.

- While for the purpose of multiplying polynomials both choices are equally good, the choice made by the electrical engineers is otherwise better. This will be explained in COMP4121 *Advanced Algorithms* in the context of JPEG compression.

$$
\begin{array}{ccccc}
P_A(x) & \xrightarrow[\Theta(n \log n)]{\text{FFT}} & \widehat{A} & & \\
 & & \times & & \\
P_B(x) & \xrightarrow[\Theta(n \log n)]{\text{FFT}} & \widehat{B} & & \Theta(n) \\
 & & = & & \\
P_C(x) & \xleftarrow[\Theta(n \log n)]{\text{IFFT}} & \widehat{C} & &
\end{array}
$$

# Summary of fast multiplication of polynomials



- We can now compute the product $P_C(x) = P_A(x)\,P_B(x)$ of two polynomials $P_A(x)$ and $P_B(x)$ in time $\Theta(n \log n)$.
- Recall that the coefficients of $P_C(x)$ are given by

$$C_t = \sum_{i+j=t} A_i B_j.$$

  The coefficient sequence $C$ is the convolution of the sequences $A$ and $B$, denoted $A \star B$.
- Therefore polynomial multiplication using the FFT allows us to find the convolution of two sequences of (up to) $n$ terms in time $\Theta(n \log n)$.

# Table of Contents



1. Introductory Examples

2. Recurrences
2.1 Framework
2.2 Master Theorem

3. Integer Multiplication
3.1 Applying D&C to multiplication of large integers
3.2 The Karatsuba trick

4. Convolutions
4.1 Polynomials
4.2 The Fast Fourier Transform

5. Self-study: Sorting and selection
5.1 Quicksort and quickselect
5.2 Median of medians

6. Puzzle

# Table of Contents



1. Introductory Examples

2. Recurrences
2.1 Framework
2.2 Master Theorem

3. Integer Multiplication
3.1 Applying D&C to multiplication of large integers
3.2 The Karatsuba trick

4. Convolutions
4.1 Polynomials
4.2 The Fast Fourier Transform

5. Self-study: Sorting and selection
5.1 Quicksort and quickselect
5.2 Median of medians

6. Puzzle

**Problem**

Let $A$ be an array of $n$ distinct, comparable elements.
Sort $A$.

**Note**

We will assume that all array elements are distinct. This simplifies some of the later reasoning, but the same ideas hold when duplicates are allowed.

### Algorithm

1. Designate the first element $x$ as the pivot.
2. Partition $A$ with all values smaller than $x$, followed by $x$ at index $j$, followed by all values larger than $x$.
3. Recurse on $A[1..(j-1)]$ and on $A[(j+1)..n]$.

### Note

We will take for granted that the second step (rearranging and partitioning the array) takes $\Theta(n)$. This is detailed in Section 7.1 of CLRS.

- In the best case, the pivot is always the median, so the subarrays each have size about $n/2$. The recurrence is $T(n) = 2T(n/2) + \Theta(n)$, so case 2 of the Master Theorem gives
$$T(n) = \Theta(n \log n).$$
- In the worst case, the pivot is always the minimum (or maximum), so one subarray is empty and the other has size $n - 1$. The recurrence is
$$T(n) = T(n-1) + \Theta(n).$$
While the Master Theorem does not apply, we can easily prove that $T(n) = \Theta(n^2)$.

- However, the worst case is very rare. If the split is $9:1$ at every step, there are only $\log_{10/9} n = O(\log n)$ many levels, each requiring a total of $O(n)$ work to partition. The overall time complexity in this case is still $\Theta(n \log n)$.

- The average case time complexity is $\Theta(n \log n)$. See Section 7.4 of CLRS for the proof.

- The worst case of the stated algorithm includes sorted arrays and reverse sorted arrays, both of which are common use cases.
- To obfuscate the worst case, one can instead use a randomly chosen pivot, or the median of the first, last and middle element.
- However, the worst case performance is still $\Theta(n^2)$ using any of these strategies.

**Note**

Since partitioning takes linear time, a pivot selection strategy that also takes $O(n)$ time doesn't worsen the overall time complexity.

# A related problem



### Selection problem

Let $A$ be an array of $n$ distinct, comparable elements, and $1 \leq k \leq n$.
Find the $k$th smallest entry of $A$.

### Attempt 1

Sort $A$ in $O(n \log n)$ using merge sort, and return $A[k]$.

### Question

Can we do better?

## Algorithm

1. Designate the first element $x$ as the pivot.
2. Partition $A$ with all values smaller than $x$, followed by $x$ at index $j$, followed by all values larger than $x$.
3. The recursive step depends on $j$.
   1. If $j = k$, return $x$.
   2. If $j > k$, recursively find the $k$th smallest element of $A[1..(j-1)]$.
   3. If $j < k$, recursively find the $(k-j)$th smallest element of $A[(j+1)..n]$.

# Complexity analysis of quickselect



- Again, the best case is when we always select the median as the pivot. The recurrence here is

$$T(n) = T(n/2) + \Theta(n),$$

  and by case 3 of the Master Theorem, $T(n) = \Theta(n)$.

- The worst case is still $\Theta(n^2)$, when we always select an extreme value as the pivot.

- Again, any split of constant proportion (e.g. 9 to 1) is sufficient to achieve $\Theta(n)$ running time.

- It can be proven that the average case complexity is $\Theta(n)$: see Section 9.2 of CLRS.

# Median



- The median is the "middle" value.
- Finding the median is a special case of the selection problem.
  - If $n$ is odd, we find the $(n+1)/2$th smallest element.
  - There are multiple conventions for the median of an array of even length. We follow CLRS by finding the $n/2$th smallest element, the "lower median".

# Finding the median



As before, we can find the median in:

- worst case $O(n \log n)$, using merge sort, or
- expected $O(n)$, using quickselect.

**Question**

Can we do better? Can we find the median (and indeed the $k$th smallest element) in worst case linear time?

**Answer**

Yes, using divide-and-conquer!

# Table of Contents



1. Introductory Examples

2. Recurrences
2.1 Framework
2.2 Master Theorem

3. Integer Multiplication
3.1 Applying D&C to multiplication of large integers
3.2 The Karatsuba trick

4. Convolutions
4.1 Polynomials
4.2 The Fast Fourier Transform

5. Self-study: Sorting and selection
5.1 Quicksort and quickselect
5.2 Median of medians

6. Puzzle

### Attempt 1

1. Divide $A$ into $n/3$ blocks of three consecutive numbers.
2. From each block, discard the smallest number and largest number, keeping only the median.
3. Recursively find the median of the remaining $n/3$ numbers, the so-called "median of medians", and return it.

### Question

Is this algorithm correct? In other words, is the median of medians the same as the median of the whole array?

### Answer

No! The original is not even guaranteed to be the median of its block.
Consider for example

$$A = [1, 3, 5, \qquad 7, 9, 2, \qquad 4, 6, 8].$$

By only keeping the median of each block, we might discard the actual median prematurely. Can we trim the search space less aggressively?

### Observations

- Suppose the median of the $n/3$ block medians is $x$.
- Half of the $n/3$ blocks have median less than $x$. In those blocks, the smallest number is even less, so it too is less than $x$.
- In the other half of the blocks, the median is greater than $x$, so the largest number is also greater than $x$.

In the following diagram, each column represents a sorted block, e.g. from an array $[84, 53, 63, 79, \ldots, 43]$.

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 53 | 45 | 21 | 8 | 11 | 19 | 27 | 3 | 43 |
| 63 | 79 | 40 | 60 | 33 | 37 | 76 | 48 | 62 |
| 84 | 83 | 70 | 88 | 98 | 51 | 82 | 96 | 69 |

For ease of visualisation, we will display the columns in sorted order of the middle row (the block medians).

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 11 | 19 | 21 | 3 | 8 | 43 | 53 | 27 | 45 |
| 33 | 37 | 40 | 48 | 60 | 62 | 63 | 76 | 79 |
| 98 | 51 | 70 | 96 | 88 | 69 | 84 | 82 | 83 |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 11 | 19 | 21 | 3 | 8 | 43 | 53 | 27 | 45 |
| 33 | 37 | 40 | 48 | 60 | 62 | 63 | 76 | 79 |
| 98 | 51 | 70 | 96 | 88 | 69 | 84 | 82 | 83 |

- The element marked in yellow is the median of medians (although it is not the overall median).
- The smaller block medians are marked in cyan.
- In these blocks, the smallest entry is also smaller than the median of medians.
- The larger block medians are marked in magenta.
- In these blocks, the largest entry is also larger than the median of medians.

## Conclusion

Ignoring rounding, there are $\frac{n}{3}$ blocks. Of these, $\frac{n}{6}$ have 'small' medians (cyan) and the other $\frac{n}{6}$ have 'large' medians (magenta). The median of medians (yellow) might not be the median of the entire array, but it is definitely:

- greater than two of the three elements in $\frac{n}{6}$ blocks, and
- less than two of the three elements in $\frac{n}{6}$ blocks.

Therefore it is greater than $2 \times \frac{n}{6} = \frac{n}{3}$ array elements and less than the same number of array elements.
So it is in the middle third of the array by value.

### Question

We can find a value in the middle third of the array. How does this help us find the median?

### Answer

Recall that quickselect has good performance as long as the pivot is not too extreme!
We will use the median of medians as a pivot selection strategy in the full selection problem, and treat median-finding as merely a special case.

## Attempt 2

1. Divide $A$ into $n/3$ blocks of three consecutive numbers.
2. Find the median of each block.
3. Recursively find the median of medians $x$, and designate it as the pivot.
4. Partition $A$ with all values smaller than $x$, followed by $x$ at index $j$, followed by all values larger than $x$.
5. The recursive step depends on $j$.
    1. If $j = k$, return $x$.
    2. If $j > k$, recursively find the $k$th smallest element of $A[1..(j-1)]$.
    3. If $j < k$, recursively find the $(k-j)$th smallest element of $A[(j+1)..n]$.

## Complexity

1. Forming the blocks takes $\Theta(n)$ time.
2. Each block has only three items, so sorting to pick the median takes constant time per block.
3. Next, we recurse on the medians of the blocks, itself a selection problem that can be solved in $T(n/3)$ time.
4. Partitioning takes $\Theta(n)$ time.
5. The median of medians is in the middle third of the partitioned array, so the last step requires us to recursively perform selection on a subarray of at most $2n/3$ elements, taking $T(2n/3)$ time.

## Complexity (continued)

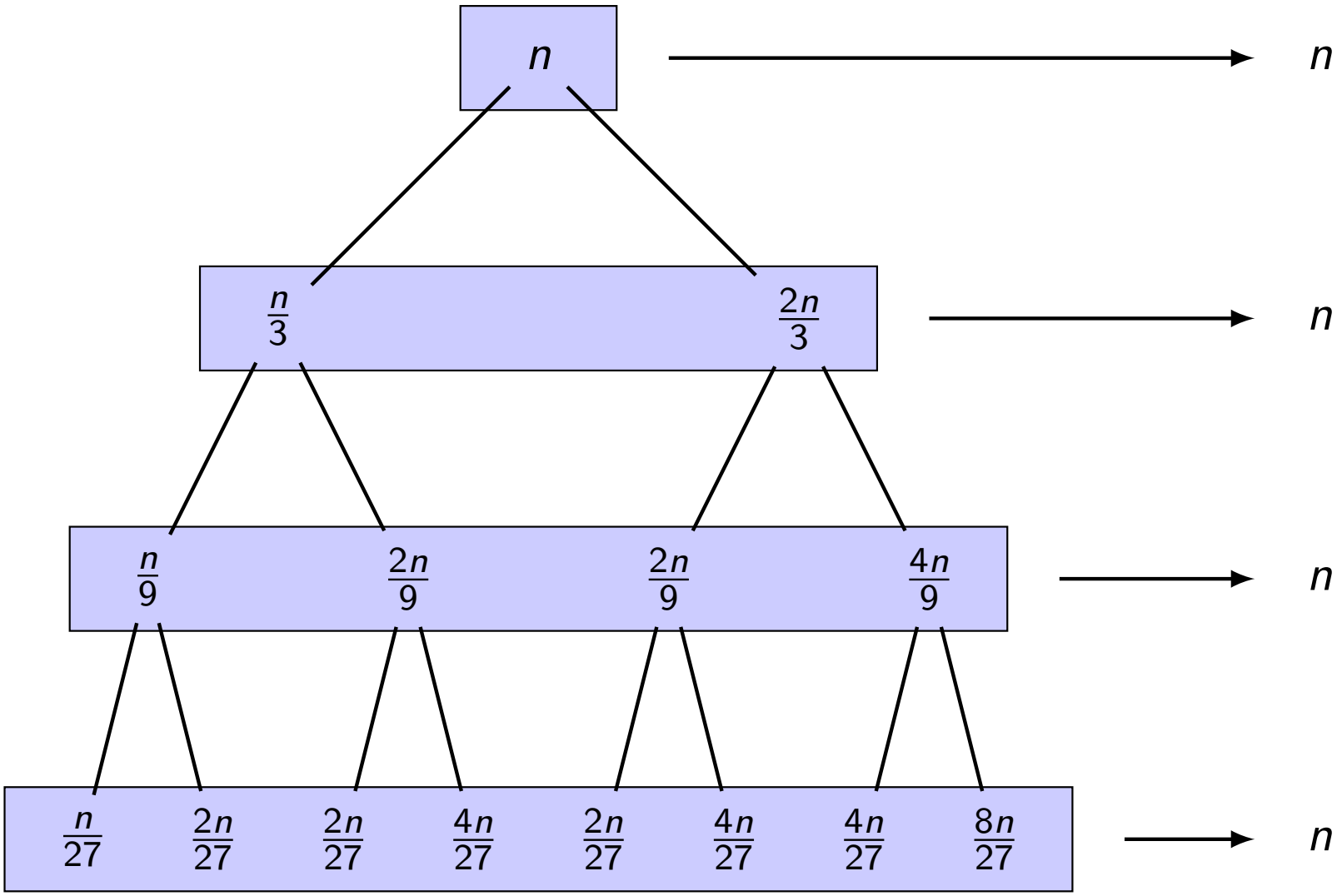
Therefore we can form the worst case recurrence

$$T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + \Theta(n).$$

Unfortunately, this is not good enough. There are $\log_{3/2} n$ levels in this recursion, and again each requires a total of $\Theta(n)$ work, for an overall time complexiy of $\Theta(n \log n)$ in the worst case.
This is asymptotically no better than just sorting! Can we do better?

# Quickselect using median of medians





# Quickselect using median of medians: block size



- One parameter of our algorithm design seemed to be an arbitrary choice: blocks of size 3.
- What happens if we increase the block size to 5?
- There will be $\frac{n}{5}$ blocks, so step 3 will recurse on $\frac{n}{5}$ medians.
- The median of medians will be larger than the median (and hence also larger than the two smallest elements) of $\frac{n}{10}$ blocks, and smaller than the median (and hence also larger than the two largest elements) of the other $\frac{n}{10}$ blocks.
- Therefore the partition we achieve is between $3:7$ and $7:3$.

In the following diagram, each column again represents a sorted block, e.g. from an array $[80, 93, 5, 70, 81, 28, \ldots, 86]$.

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 5 | 22 | 17 | 3 | 1 | 6 | 19 | 7 | 15 |
| 70 | 28 | 32 | 49 | 23 | 57 | 59 | 21 | 26 |
| 80 | 44 | 46 | 60 | 36 | 58 | 82 | 35 | 27 |
| 81 | 63 | 62 | 73 | 68 | 64 | 90 | 69 | 86 |
| 93 | 94 | 87 | 89 | 91 | 95 | 96 | 77 | 92 |

For ease of visualisation, we will display the columns in sorted order of the middle row (the block medians).

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 15 | 7 | 1 | 22 | 17 | 6 | 3 | 5 | 19 |
| 26 | 21 | 23 | 28 | 32 | 57 | 49 | 70 | 59 |
| 27 | 35 | 36 | 44 | 46 | 58 | 60 | 80 | 82 |
| 86 | 69 | 68 | 63 | 62 | 64 | 73 | 81 | 90 |
| 92 | 77 | 91 | 94 | 87 | 95 | 89 | 93 | 96 |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 15 | 7 | 1 | 22 | 17 | 6 | 3 | 5 | 19 |
| 26 | 21 | 23 | 28 | 32 | 57 | 49 | 70 | 59 |
| 27 | 35 | 36 | 44 | 46 | 58 | 60 | 80 | 82 |
| 86 | 69 | 68 | 63 | 62 | 64 | 73 | 81 | 90 |
| 92 | 77 | 91 | 94 | 87 | 95 | 89 | 93 | 96 |

- The element marked in yellow is the median of medians (although it is not the overall median).
- The smaller block medians are marked in cyan.
- In these blocks, the smallest two entries are also smaller than the median of medians.
- The larger block medians are marked in magenta.
- In these blocks, the largest two entries are also larger than the median of medians.

## Conclusion

Ignoring rounding, there are $\frac{n}{5}$ blocks. Of these, $\frac{n}{10}$ have 'small' medians (cyan) and the other $\frac{n}{10}$ have 'large' medians (magenta). The median of medians (yellow) is definitely:

- greater than three of the five elements in $\frac{n}{10}$ blocks, and
- less than three of the five elements in $\frac{n}{10}$ blocks.

Therefore it is greater than $3 \times \frac{n}{10}$ array elements and less than the same number of array elements.
So it is in the middle two-fifths of the array by value.

## Algorithm

1. Divide $A$ into $n/5$ blocks of five consecutive numbers.
2. Find the median of each block.
3. Recursively find the median of medians $x$.
4. Partition $A$ with all values smaller than $x$, followed by $x$ at index $j$, followed by all values larger than $x$.
5. The recursive step depends on $j$.
    1. If $j = k$, return $x$.
    2. If $j > k$, recursively find the $k$th smallest element of $A[1..(j-1)]$.
    3. If $j < k$, recursively find the $(k-j)$th smallest element of $A[(j+1)..n]$.

## Complexity

1. Forming the blocks takes $\Theta(n)$ time.
2. Each block has only five items, so sorting to pick the median takes constant time per block.
3. Next, we recurse on the medians of the blocks, itself a selection problem that can be solved in $T(n/5)$ time.
4. Partitioning takes $\Theta(n)$ time.
5. The median of medians is between the third and seventh deciles of $A$, so we recursively select from a subarray of at most $7n/10$ elements, taking $T(7n/10)$ time.

## Complexity (continued)

Therefore we can form the worst case recurrence

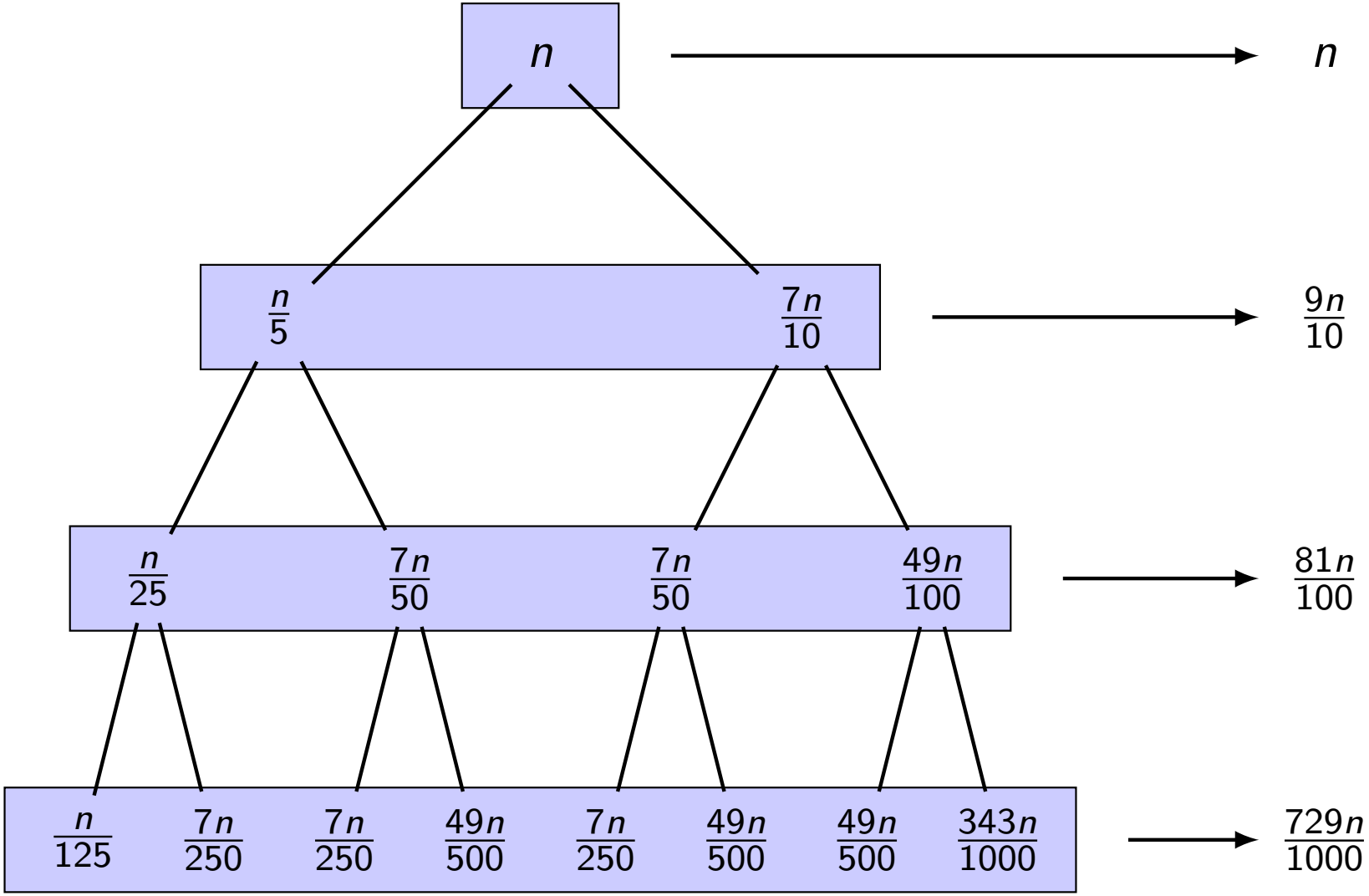
$$T(n) = T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + \Theta(n).$$

Surprisingly, this subtle change is enough to reduce the worst case time complexity from $\Theta(n \log n)$ to $\Theta(n)$.
The crucial difference is that

$$\frac{1}{3} + \frac{2}{3} = 1 > \frac{1}{5} + \frac{7}{10}.$$

# Quickselect using median of medians: blocks of five





## Complexity (continued)

- Each subproblem requires us to form and sort blocks and then partition a subarray, taking time linear in the number of elements.
- We can therefore group multiple subproblems, which together will take time linear in their total number elements.
- The subarrays considered in the $k$th level have a total of $\left(\frac{9}{10}\right)^{k-1} n$ elements.
- Adding up over all levels, we get a geometric series with sum at most $10n$.
- Therefore we have succeeded in solving the selection problem in worst case linear time!

The same pivot selection strategy allows us to develop a corresponding variant of quicksort.

**Algorithm**

1. Find the median $x$ using median of medians quickselect, and designate it as the pivot.
2. Partition $A$ with all values smaller than $x$, followed by $x$ at index $j$, followed by all values larger than $x$.
3. Recurse on $A[1..(j-1)]$ and on $A[(j+1)..n]$.

## Complexity

- The first two steps each take linear time, and we are guaranteed to recurse into two subproblems each of size $n/2$.
- Therefore the recurrence is

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n),$$

and by case 2 of the Master Theorem, the worst case time complexity is $\Theta(n \log n)$.

### Question

Is this better than the original quicksort?

### Answer

Yes and no.

- There is significant overhead involved in finding the median, so the constant factor hidden by the big-oh notation is substantial.
- In many practical applications this outweighs the improvement on the worst case.

# Table of Contents



1. Introductory Examples

2. Recurrences
2.1 Framework
2.2 Master Theorem

3. Integer Multiplication
3.1 Applying D&C to multiplication of large integers
3.2 The Karatsuba trick

4. Convolutions
4.1 Polynomials
4.2 The Fast Fourier Transform

5. Self-study: Sorting and selection
5.1 Quicksort and quickselect
5.2 Median of medians

6. Puzzle

## Problem

Five pirates have to split 100 bars of gold. They all line up and proceed as follows:

- The first pirate in line gets to propose a way to split up the gold (for example: everyone gets 20 bars).
- The pirates, including the one who proposed, vote on whether to accept the proposal. If the proposal is rejected, the pirate who made the proposal has to walk the plank.
- The next pirate in line then makes his proposal, and the 4 pirates vote again. If the vote is tied (2 vs 2) then the proposing pirate still has to walk the plank. Only a majority can accept a proposal.

This process continues until a proposal is accepted or there is only one pirate left.

### Problem (continued)

Assume that every pirate has the same priorities, in the following order:

1. not having to walk the plank;
2. getting as much gold as possible;
3. seeing other pirates walk the plank, just for fun.

### Problem (continued)

What proposal should the first pirate make?

### Hint

Assume first that there are only two pirates, and see what happens. Then assume that there are three pirates and that they have figured out what happens if there were only two pirates and try to see what they would do. Further, assume that there are four pirates and that they have figured out what happens if there were only three pirates, try to see what they would do. Finally, assume there are five pirates and that they have figured out what happens if there were only four pirates.

**That's All, Folks!!**